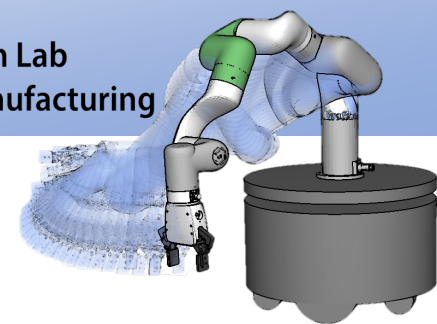


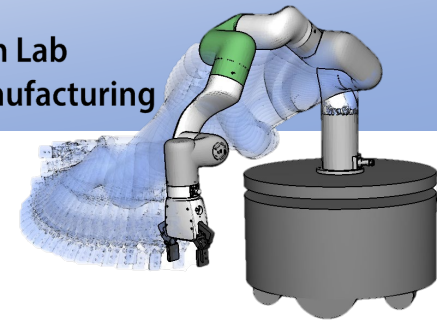


双方向リスト

知能システムコース 准教授 万 偉偉

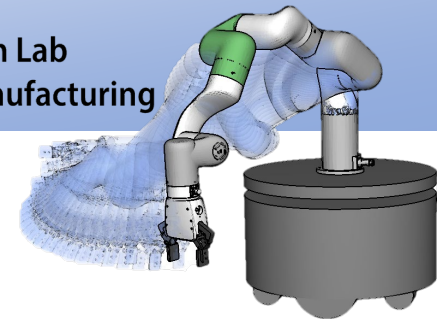
基礎工学部 F522号室

wan@sys.es.osaka-u.ac.jp



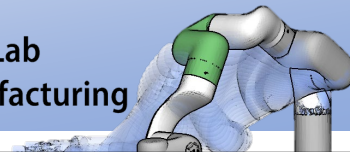
復習 ポインタ

- ポインタ
 - 変数の値が格納されている記憶装置の位置を指す
 - 左辺値を右辺値として持つ
- ポインタの演算
 - 代入
 - 参照
 - 前進・後進
 - 等価
- ポインタは計算機アドレスと異なり，強く型に縛られている



復習 構造体

- 宣言 : struct 構造体タグ名 変数名
- ポインタ宣言 : struct 構造体タグ名 *変数名
- メンバの参照
- 構造体の配列
- 構造体ポインタのメモリ管理
 - void *malloc(size_t size)
 - void free(void *ptr)
- 構造体宣言の簡略
 - typedef struct 構造体タグ名 変数名



復習

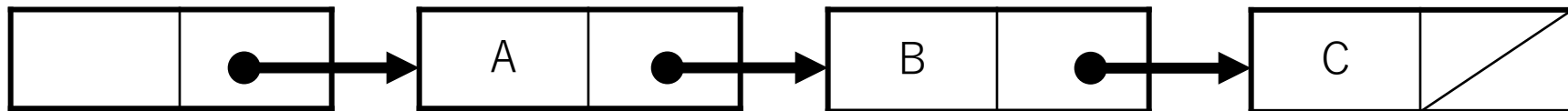
・リスト

```

struct CELL *create(DATATYPE data) {
    struct CELL *p = (struct CELL *)malloc(sizeof(struct CELL));
    p->next = NULL;
    p->data = data;
    return p;
}
  
```

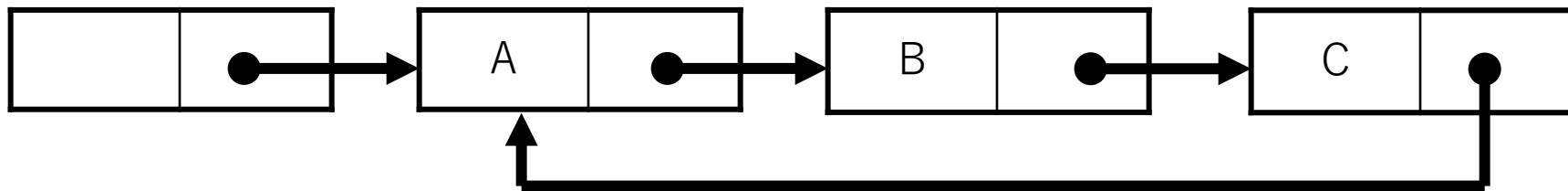
連結リスト linked list

header

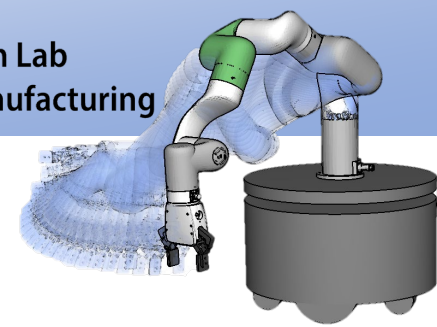


循環リスト circular list

header



後ろに移動し続ければ、先頭に戻る



構造体とポインタを用いた 連結リストの実装

```
struct CELL *create(DATATYPE data) {  
    struct CELL *p =  
        p->next = NULL;  
        p->data = data;  
    return p;  
}
```

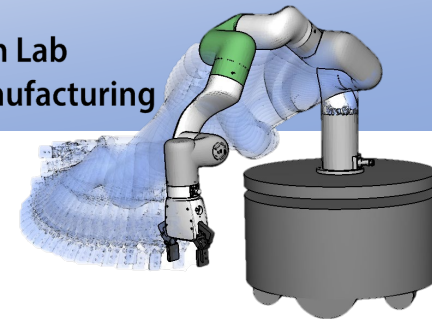
()malloc(sizeof());

struct CELL の格納領域を
ヒープ領域に確保する

間違ったコーディング例)

```
struct CELL *create(DATATYPE data) {  
    struct CELL p; /* 局所変数 */  
    p.next = NULL;  
    p.data = data;  
    return &p;  
}
```

```
struct CELL create(DATATYPE data) {  
    struct CELL p; /* 局所変数 */  
    p.next = NULL;  
    p.data = data;  
    return p;  
}
```



復習

• スタック LIFO

スタックの最大容量 →

stack[MAX-1]	
...	
stack[N]	
stack[N-1]	
...	
stack[3]	
stack[2]	
stack[1]	
stack[0]	

スタックの top →

疑似コードによる ADT Stack の実装

Push: $\text{stack}[\text{N}++] \leftarrow \text{data}$ if $\text{N} < \text{MAX}$

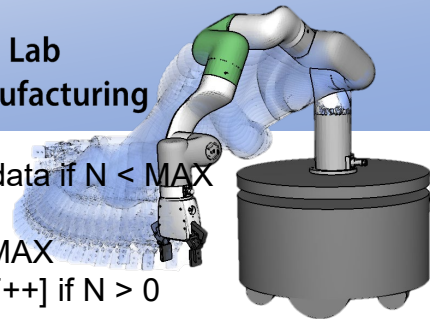
Pop: $\text{data} \leftarrow \text{stack}[\text{--N}]$ if $\text{N} > 0$

Top: $\text{stack}[\text{N}-1]$ if $\text{N} > 0$

isEmpty: true if $\text{N} \leq 0$

otherwise false

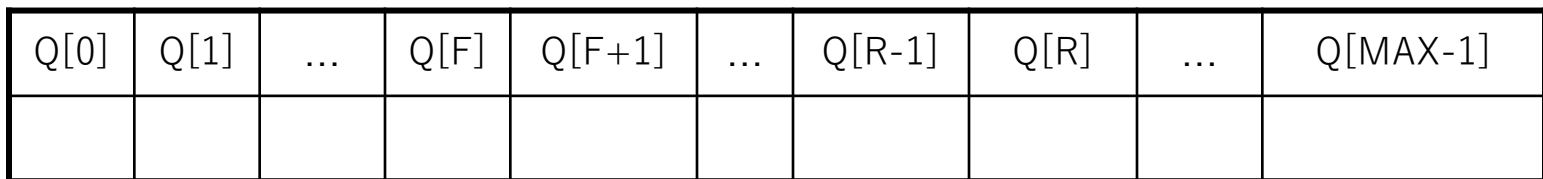
Size: N



復習

• 待ち行列（キュー）FIFO

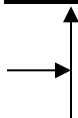
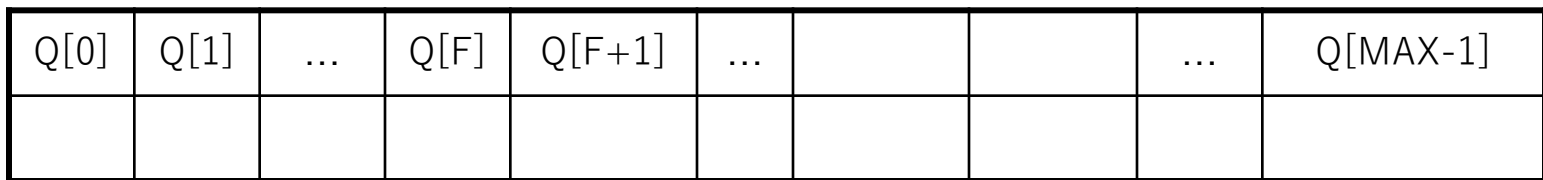
Enqueue: $Q[R++] \leftarrow \text{data}$ if $N < \text{MAX}$
 $N \leftarrow N+1$
 $R \leftarrow 0$ if $R \geq \text{MAX}$
 Dequeue: $\text{data} \leftarrow Q[F++]$ if $N > 0$
 $N \leftarrow N-1$
 $F \leftarrow 0$ if $F \geq \text{MAX}$
 Front: $Q[F]$ if $N > 0$
 isEmpty: true if $N \leq 0$
 otherwise false



← Queueの要素数 N →

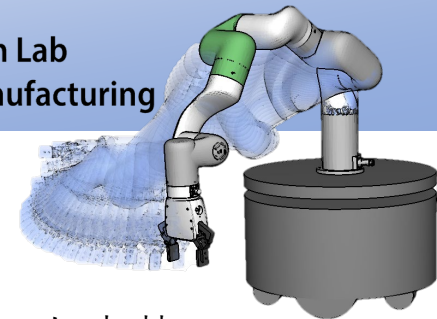
Queueの先頭

Queueの最後尾



Queueの要素数 N

Queueの最後尾 R Queueの先頭 F



復習

• 再帰

if $n == 0$ return constant
else return $f(n-1)$

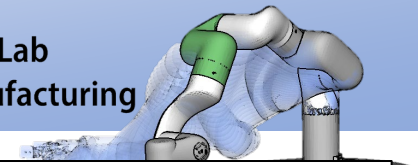
- 終止条件
- 繰り返し呼び出す
- 繰り返し作業で効率化

```
int fib(int n) {
    if (n <= 1)
        return n;
    else
        return fib(n - 1) + fib(n - 2);
}
```

```
int fib(int n) {
    int a = 0, b = 1, temp, i;
    for (i = 2; i <= n; i++) {
        temp = a + b;
        a = b;
        b = temp;
    }
    return (n == 0) ? a : b;
}
```

スタックを用いた実装

```
#define MAX_N 100
int stack[MAX_N];
int top = -1;
void push(int val) {stack[++top] = val;}
int pop() {return stack[top--];}
int fib(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    push(0); // fib(0)
    push(1); // fib(1)
    for (int i = 2; i <= n; i++) {
        int b = pop(); // fib(i-1)
        int a = pop(); // fib(i-2)
        int c = a + b; // fib(i)
        push(b); // 再び使うため
        push(c); // 最新のfib(i)
    }
    return pop(); // fib(n)}
}
```



復習

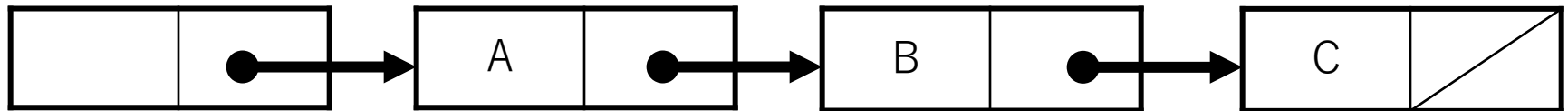
・リスト

```

struct CELL *create(DATATYPE data) {
    struct CELL *p = (struct CELL *)malloc(sizeof(struct CELL ));
    p->next = NULL;
    p->data = data;
    return p;
}
  
```

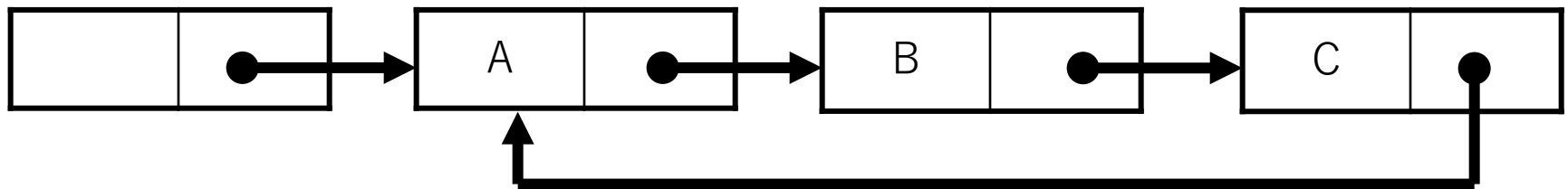
連結リスト linked list

header

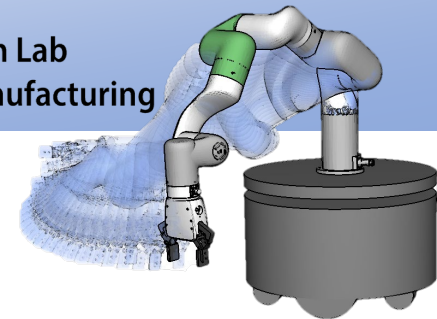


循環リスト circular list

header



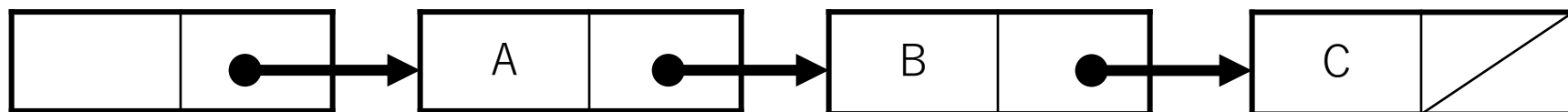
後ろに移動し続ければ、先頭に戻る



循環リスト

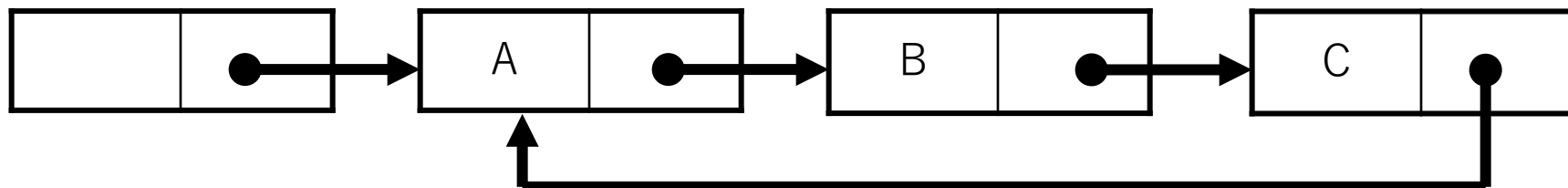
連結リスト linked list

header

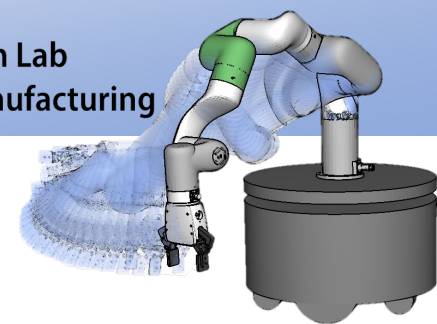


循環リスト circular list

header

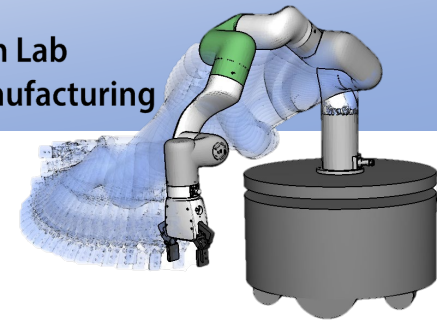


後ろに移動し続ければ、先頭に戻る



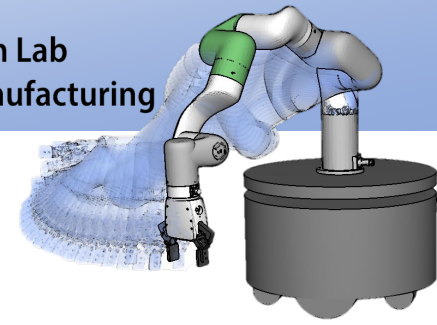
循環リストの特徴と目的

- 環状に並んだデータ構造を表現
- 要素を結び合わせるポインタをたどれば、循環リストに含まれる全要素を順番に処理できる
- 厳密には「最初」と「最後」の要素というものは存在しない



循環リストの操作

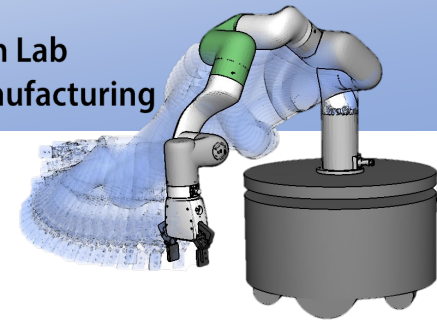
- 要素の挿入・削除
 - 連結リストの場合と同様
- 要素の追跡(trace)
 - 基本的に連結リストと同様
 - 最後尾がNULLでは判定できないので工夫が必要→追跡を開始した要素に到達したら終了



要素追跡の実装例

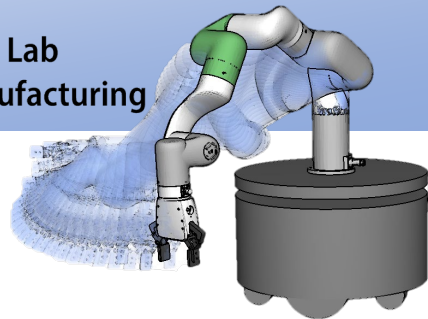
```
struct CELL *p, *ptr; /* 追跡用のポインタ変数 */  
  
if (ptr != NULL) { /* 追跡開始点 ptr */  
    p = ptr; /* 追跡開始点を作業用変数 p にコピー */  
    do {  
        /* p が指すセルの処理 */  
        p = p->next;  
    } while (p != ptr); /* 開始点まで戻ってきたら終了 */  
}
```

変数 ptr が NULL なら，この循環リストは空なので何もしない。
リストが空でないなら，セルを順番にたどって処理する
Do .. While 形式によりループ末尾で終了条件を判断する



連結リストの問題点

- 各セルは次のセルへのポインタ **のみ** を持つ。
 - セル間の移動は 1 方向のみ
- 前後に自由に移動できるリスト構造
 - Doubly-linked list (双方向連結リスト, 双方向リスト, 2重連結リスト)

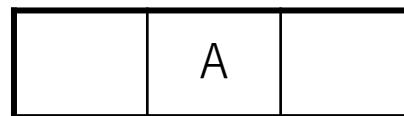


双方向リスト doubly-linked list

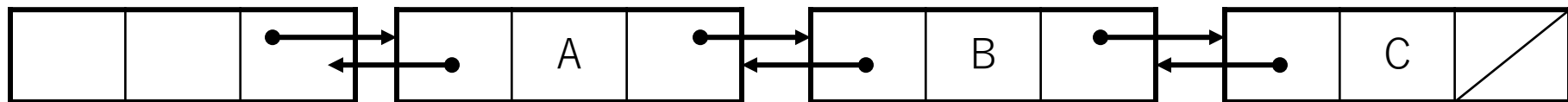
Linked list

```

struct CELL {
  char data;
  struct CELL * next;
};
  
```



header

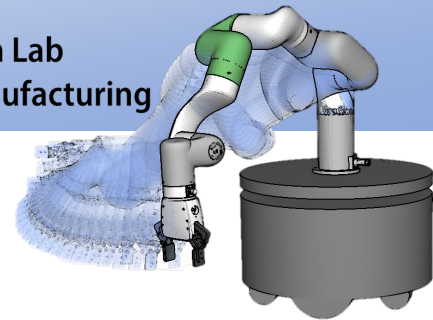


Doubly linked list

- 各セルは2つのポインタを持つ
 - 次のセルへのポインタ
 - 前のセルへのポインタ
- 前後のセルへのポインタを持っているので前後に移動可能

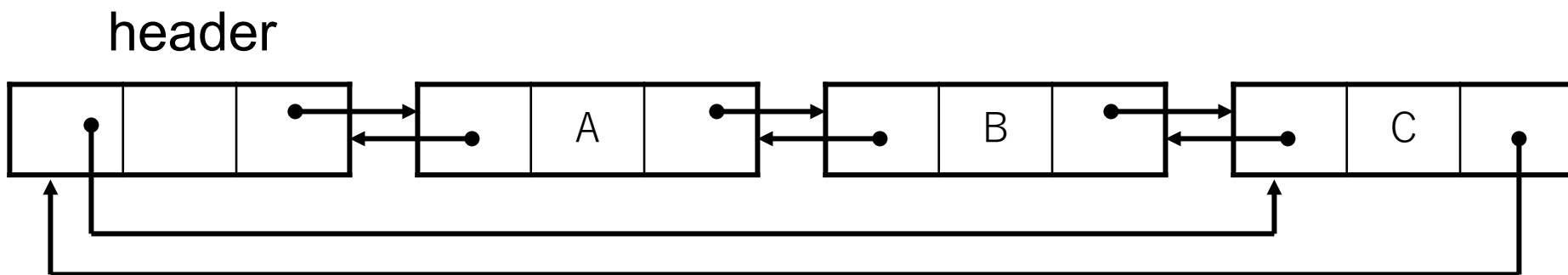
```

struct CELL {
  char data;
  struct CELL* prev;
  struct CELL* next;
};
  
```



双方向リスト doubly-linked list

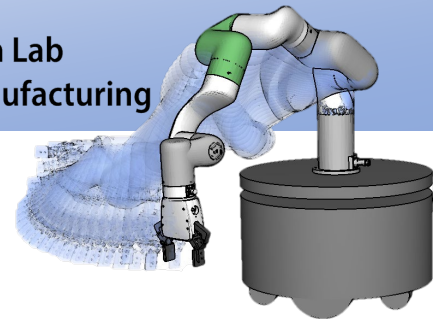
リストの末尾も分かっていると tail() などの実装が容易



header を利用する

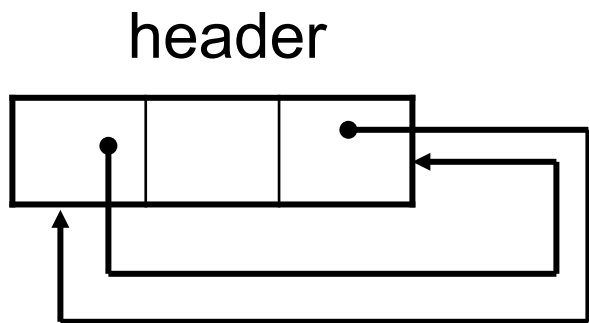
最後のセルが header を指すようにする

header の前のセルを指すポインタが最後のセルを指すようにする

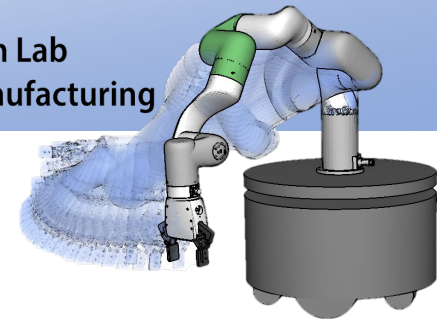


双方向リスト doubly-linked list

空リストの表現



header の前後のポインタが header 自身を指す.



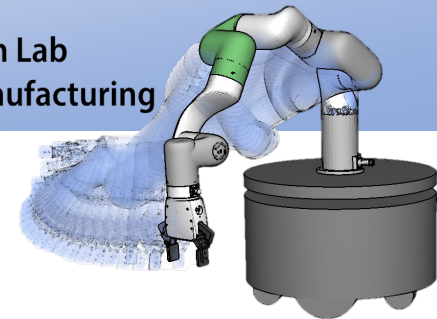
双方向リストの利点

- 利点

- 双方向リストでは、セルは前後のポインタを持っているため、制約を受けず挿入、削除などの操作が可能
- 一方、連結リストでは、一般に挿入、削除などの操作に制約がある.

- 例) 挿入の場合

- 連結リストでは、指定したセルの直後には挿入できるが直前には挿入できない



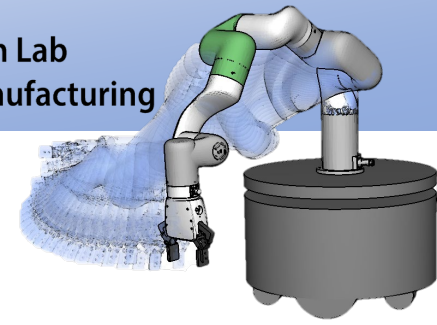
双方向リストの欠点

- 連結リストと比べるとポインタを1つ余計に持つ必要がある
- 取り扱うデータが小さい場合（例えば、整数），2個のポインタを持つため記憶容量に対してオーバーヘッドが大きい（データサイズが大きい場合は問題ではない）

トレードオフ問題

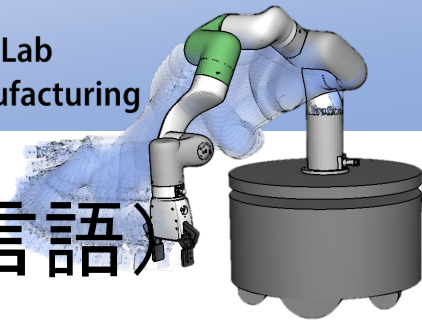
操作の自由さ ⇔ 余分なデータサイズ

最近では、記憶容量が向上してきたので操作の自由さ（設計の自由さ）が優先されつつある



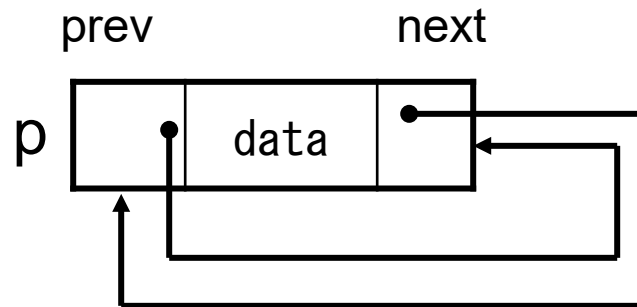
双方向リストのセル（C言語）

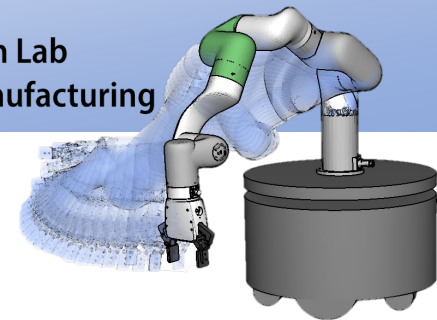
```
struct CELL {  
    struct CELL *prev; /* 前のデータへのポインタ */  
    struct CELL *next; /* 次のデータへのポインタ */  
    DATATYPE data;     /* 格納されるデータ */  
};
```



空リスト（要素）生成の実装（C言語）

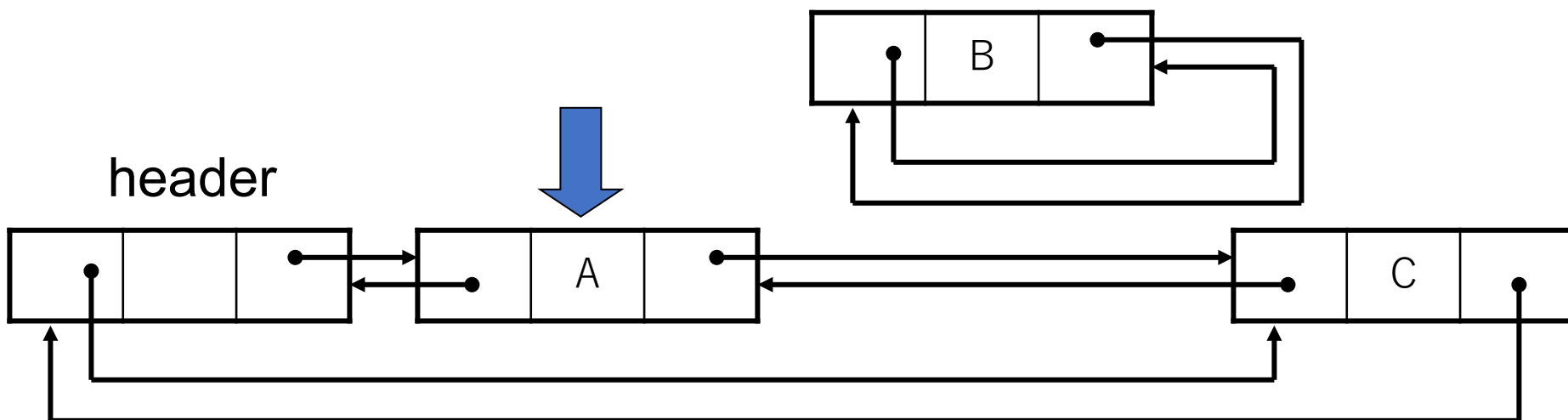
```
struct CELL *create(DATATYPE data) {  
    struct CELL *p = (struct CELL*)malloc(sizeof(struct CELL));  
    p->prev = p;  
    p->next = p;  
    p->data = data;  
    return p;  
}
```

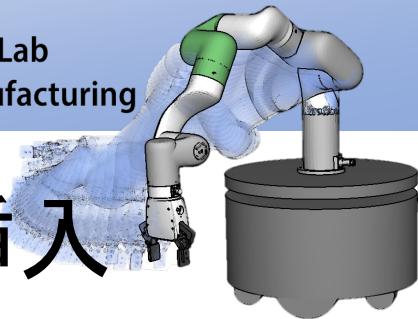




要素挿入の実装

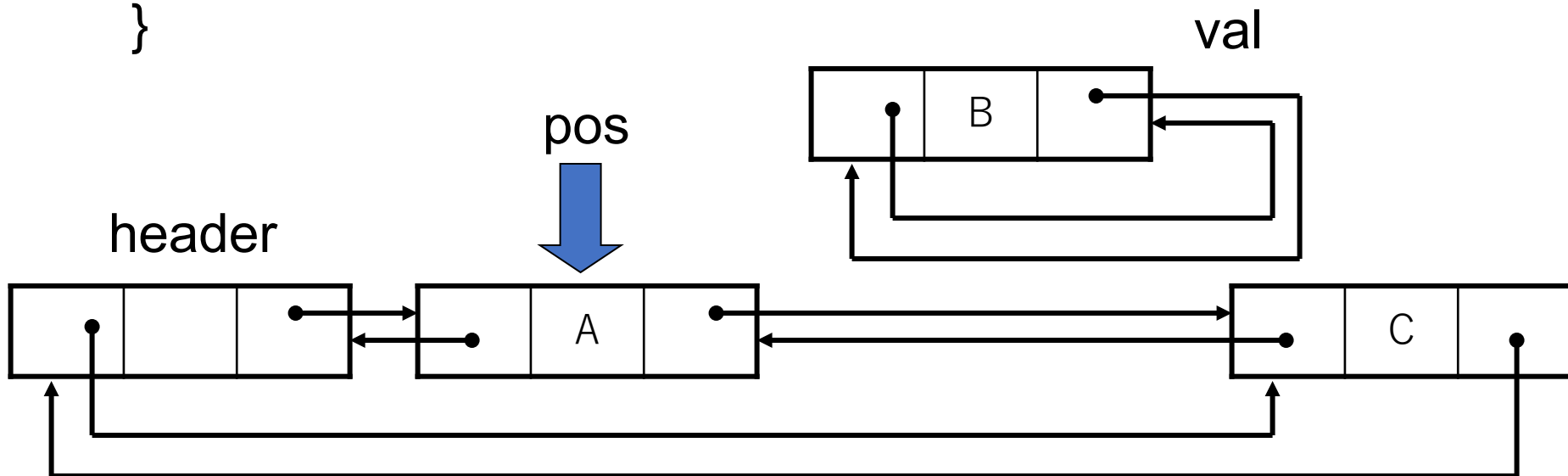
- 指定された要素の次の位置に挿入
 - 連結リストとほぼ同様
- 指定された要素の前の位置に挿入
 - 前のセルに移動して、次の要素に挿入

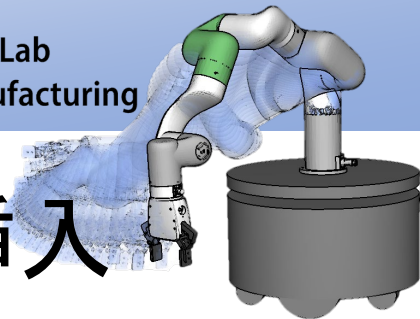




指定された要素の次の位置に挿入

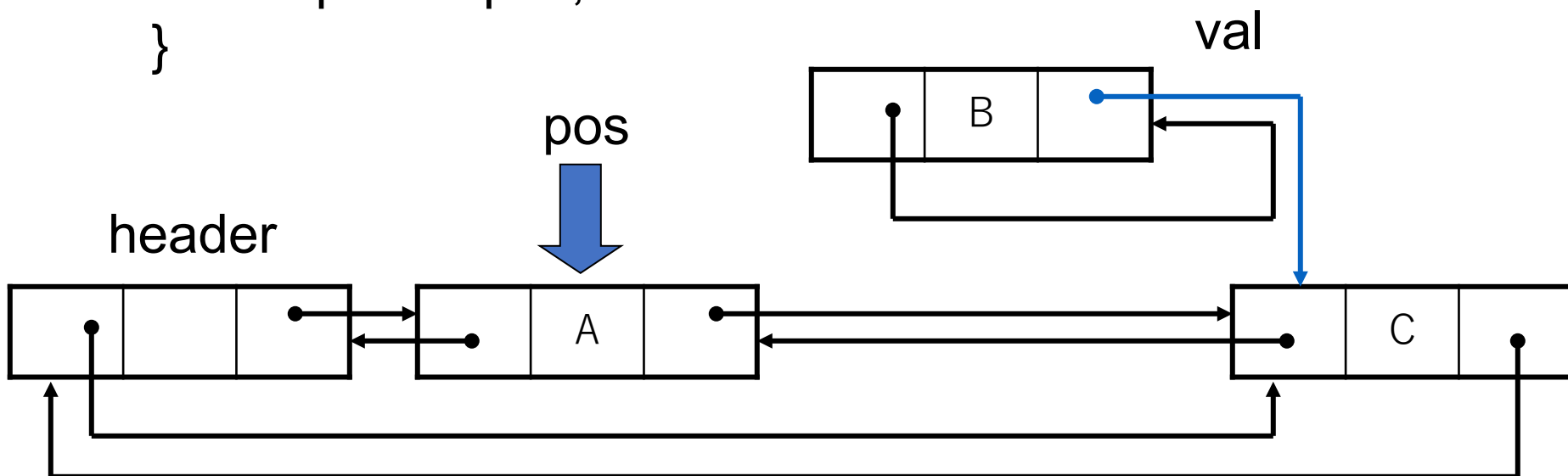
```
void insert( struct CELL *pos, struct CELL *val ) {
    val->next = pos->next;
    val->next->prev = val;
    pos->next = val;
    val->prev = pos;
}
```

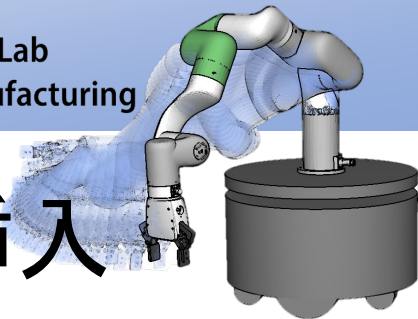




指定された要素の次の位置に挿入

```
void insert( struct CELL *pos, struct CELL *val ) {  
    val->next = pos->next;  
    val->next->prev = val;  
    pos->next = val;  
    val->prev = pos;  
}
```

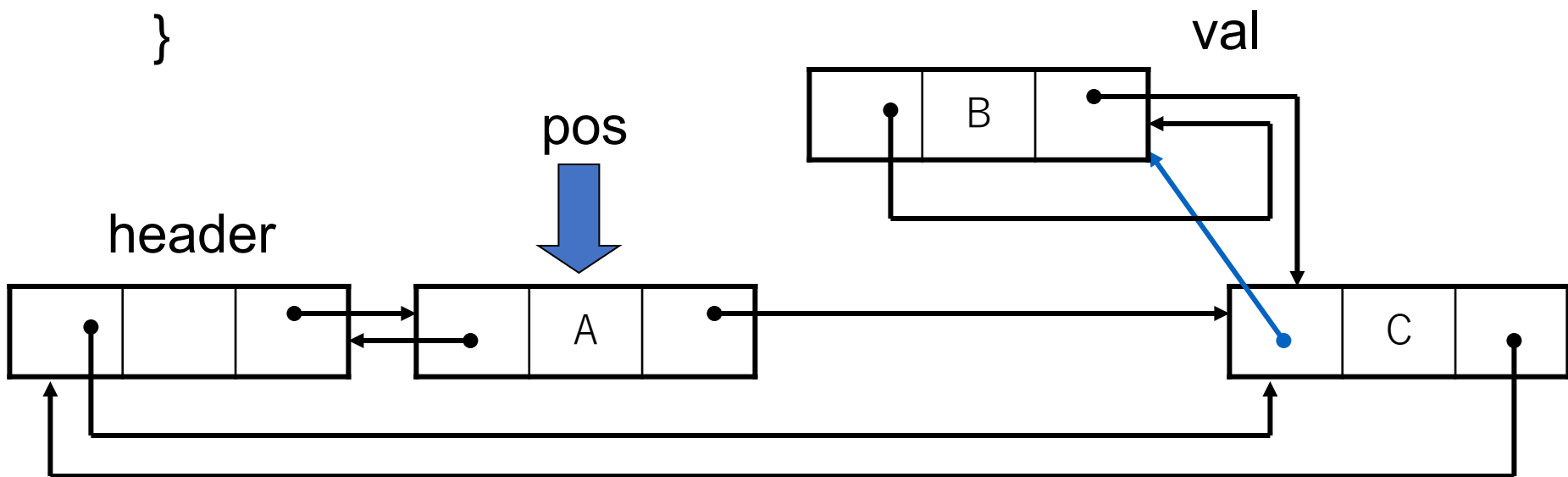


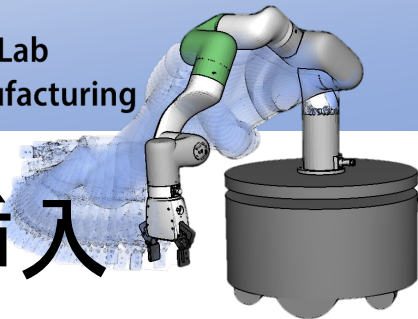


指定された要素の次の位置に挿入

```

void insert( struct CELL *pos, struct CELL *val ) {
    val->next = pos->next;
    val->next->prev = val;
    pos->next = val;
    val->prev = pos;
}
  
```

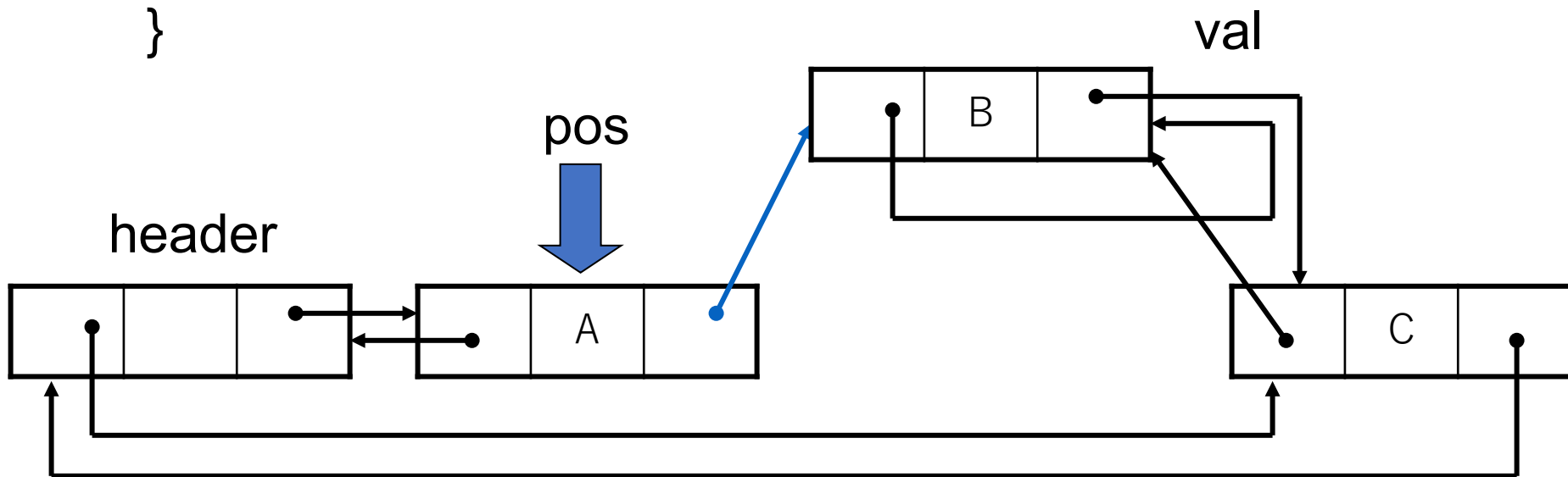


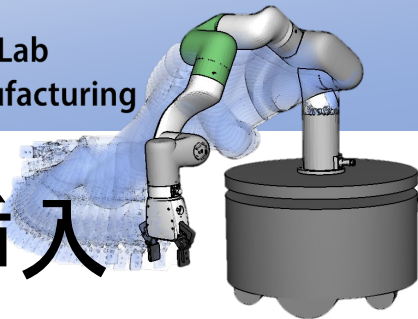


指定された要素の次の位置に挿入

```

void insert( struct CELL *pos, struct CELL *val ) {
    val->next = pos->next;
    val->next->prev = val;
    pos->next = val;
    val->prev = pos;
}
  
```

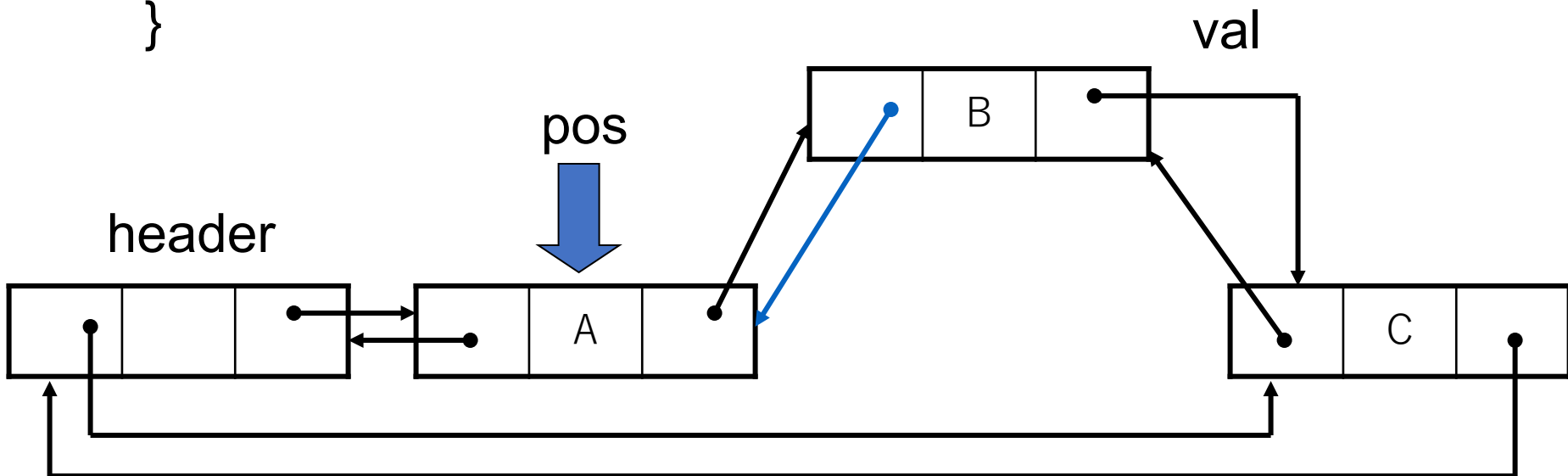


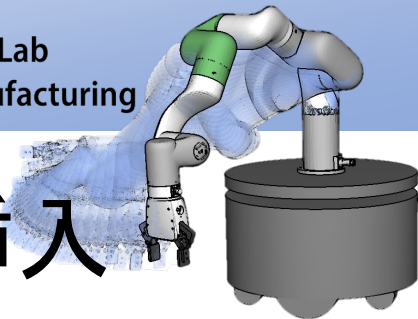


指定された要素の次の位置に挿入

```

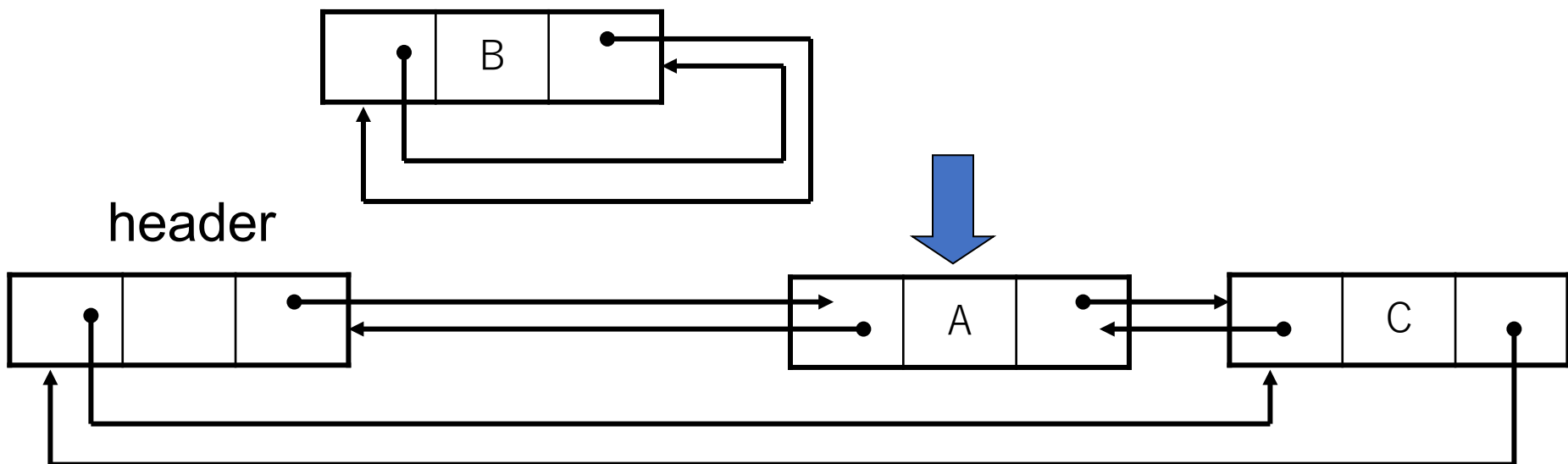
void insert( struct CELL *pos, struct CELL *val ) {
    val->next = pos->next;
    val->next->prev = val;
    pos->next = val;
    val->prev = pos;
}
    
```

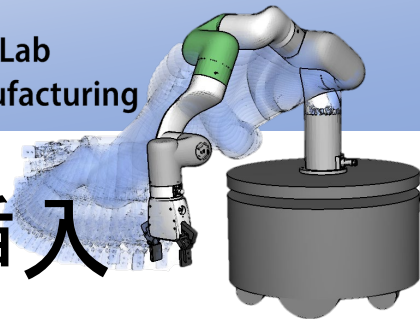




指定された要素の前の位置に挿入

- 前のセルに移動して，次の要素に挿入





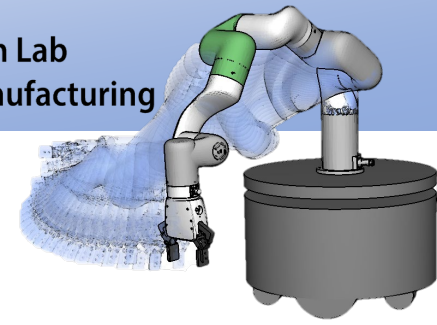
指定された要素の前の位置に挿入

- 前のセルに移動して， 次の要素に挿入

```
void insert_before( struct CELL *pos, struct CELL *val ) {  
    insert( pos->prev, val );  
}
```

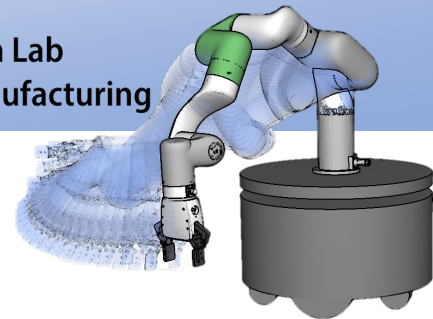
挿入ポイントを前のセルに移動

```
void insert( struct CELL *pos, struct CELL *val ) {  
    val->next = pos->next;  
    val->next->prev = val;  
    pos->next = val;  
    val->prev = pos;  
}
```



要素削除の実装

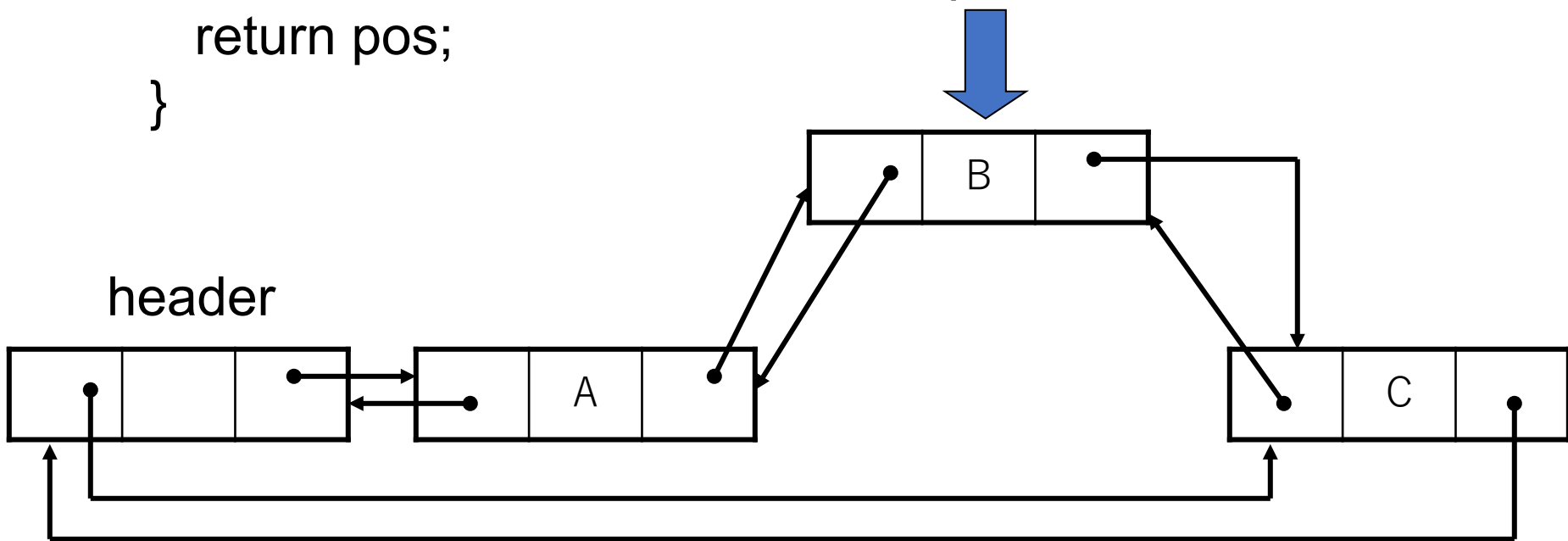
- 指定された要素の次を削除
 - 連結リストとほぼ同様
- 指定された要素を削除する

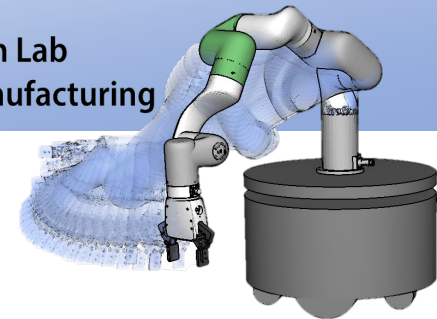


指定された要素を削除する

```

struct CELL *erase( struct CELL *pos ) {
    pos->prev->next = pos->next;
    pos->next->prev = pos->prev;
    pos->prev = pos->next = pos;
    return pos;
}
  
```

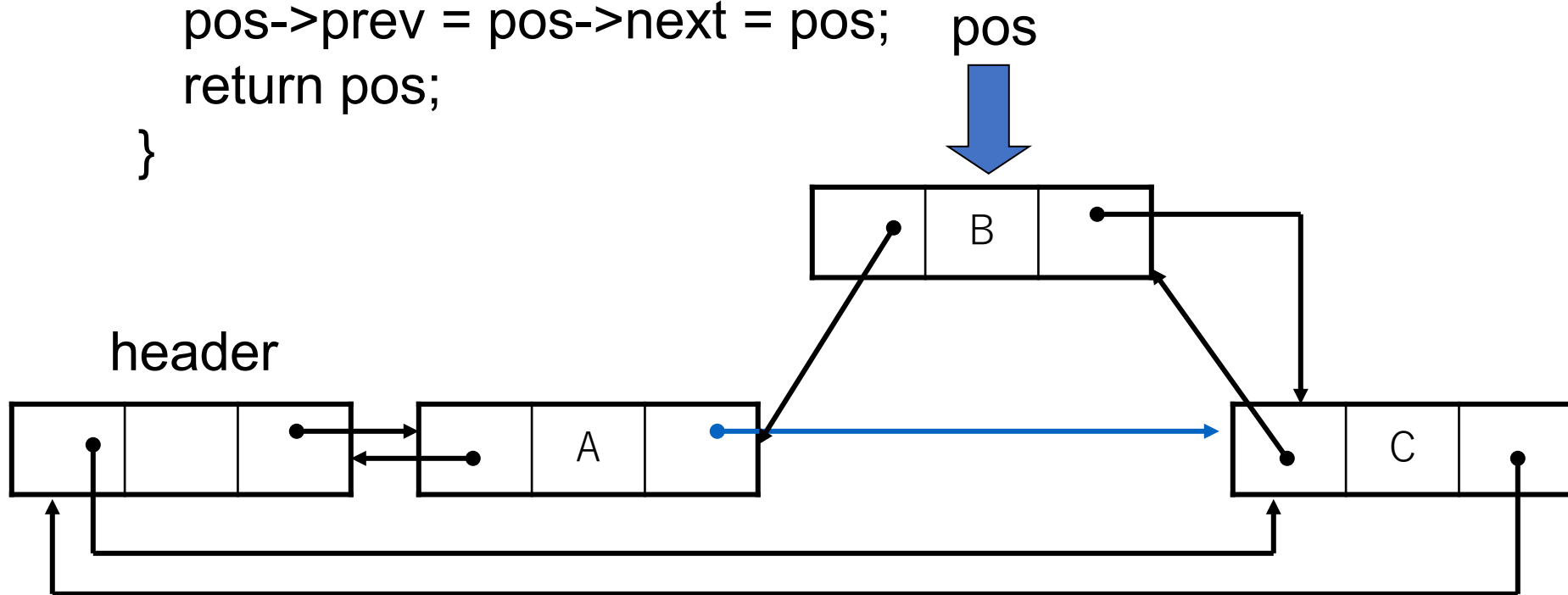


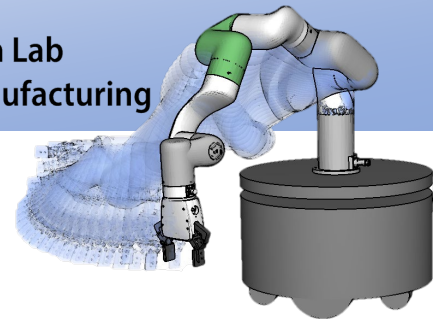


指定された要素を削除する

```

struct CELL *erase( struct CELL *pos ) {
  pos->prev->next = pos->next;
  pos->next->prev = pos->prev;
  pos->prev = pos->next = pos;
  return pos;
}
  
```

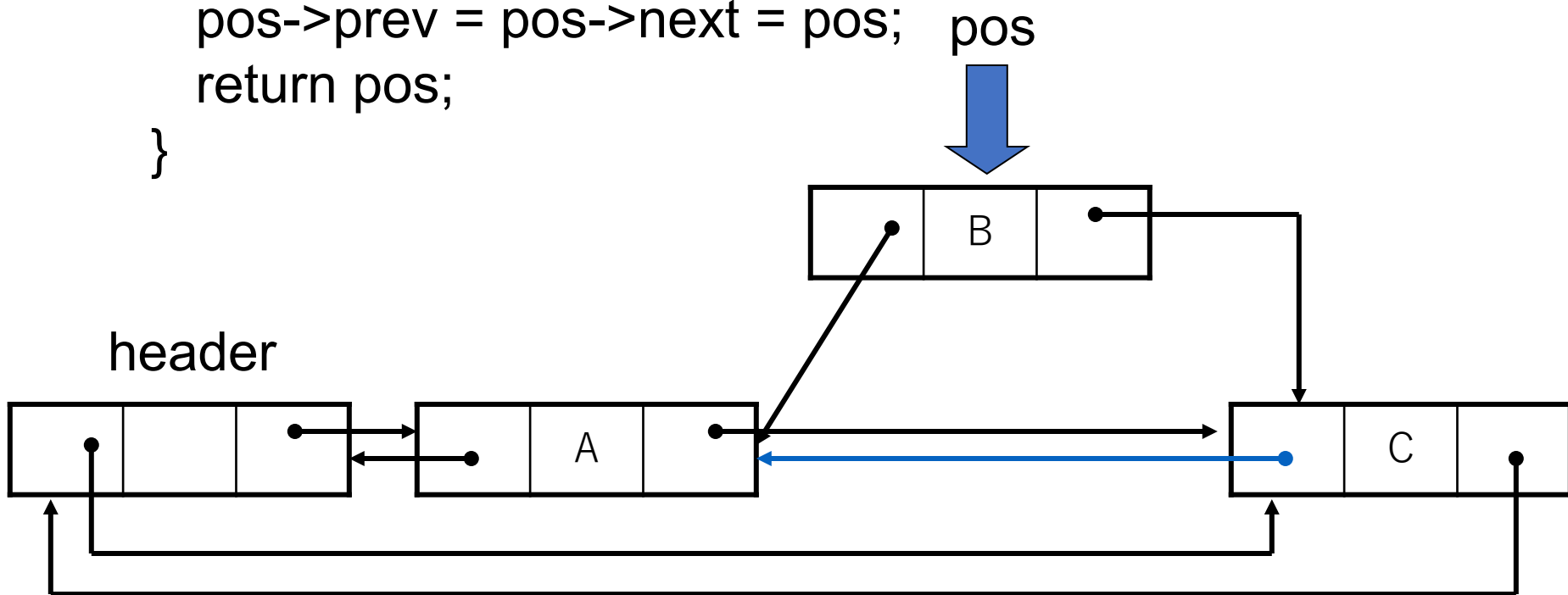


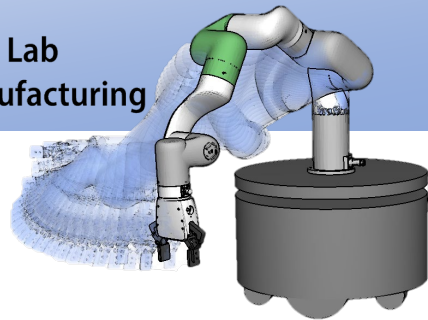


指定された要素を削除する

```

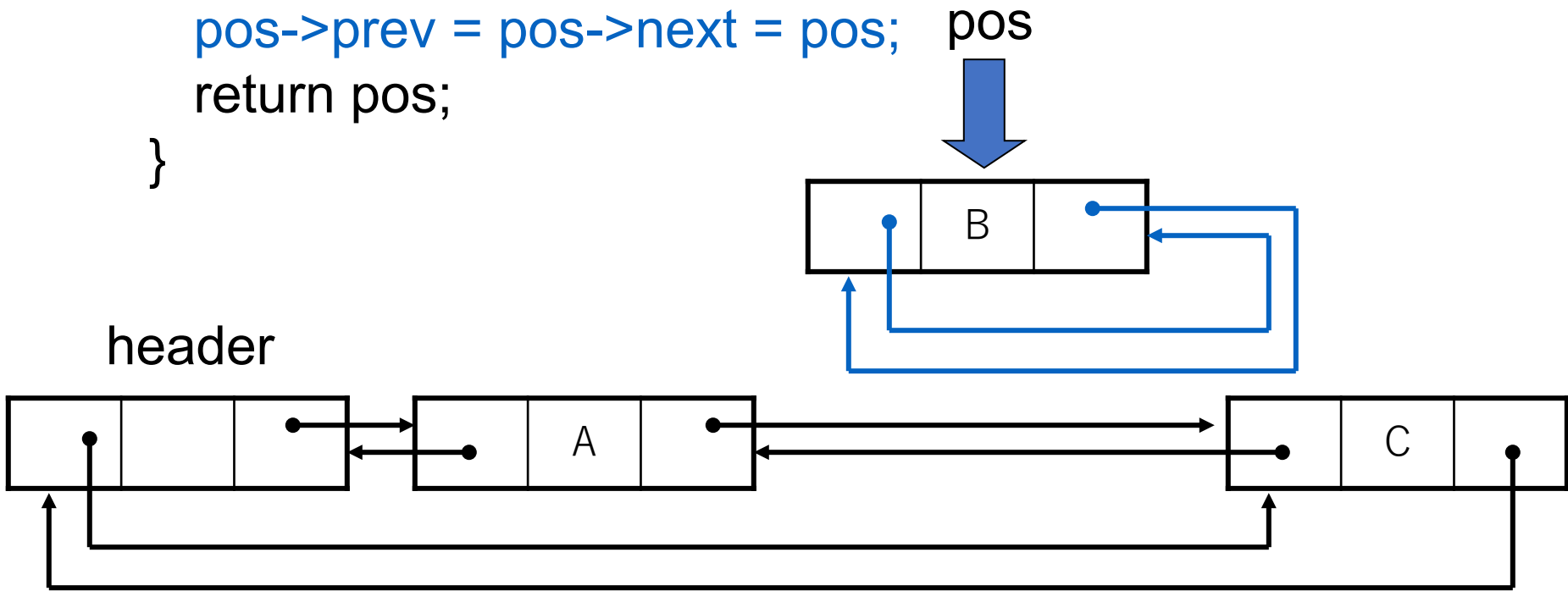
struct CELL *erase( struct CELL *pos ) {
    pos->prev->next = pos->next;
    pos->next->prev = pos->prev;
    pos->prev = pos->next = pos;
    return pos;
}
  
```

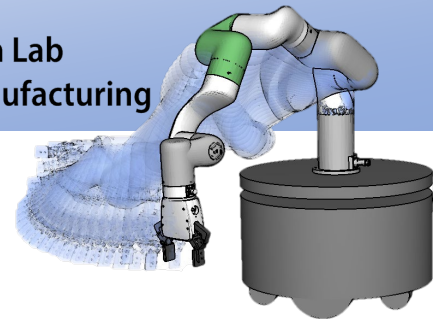




指定された要素を削除する

```
struct CELL *erase( struct CELL *pos ) {
    pos->prev->next = pos->next;
    pos->next->prev = pos->prev;
    pos->prev = pos->next = pos;
    return pos;
}
```

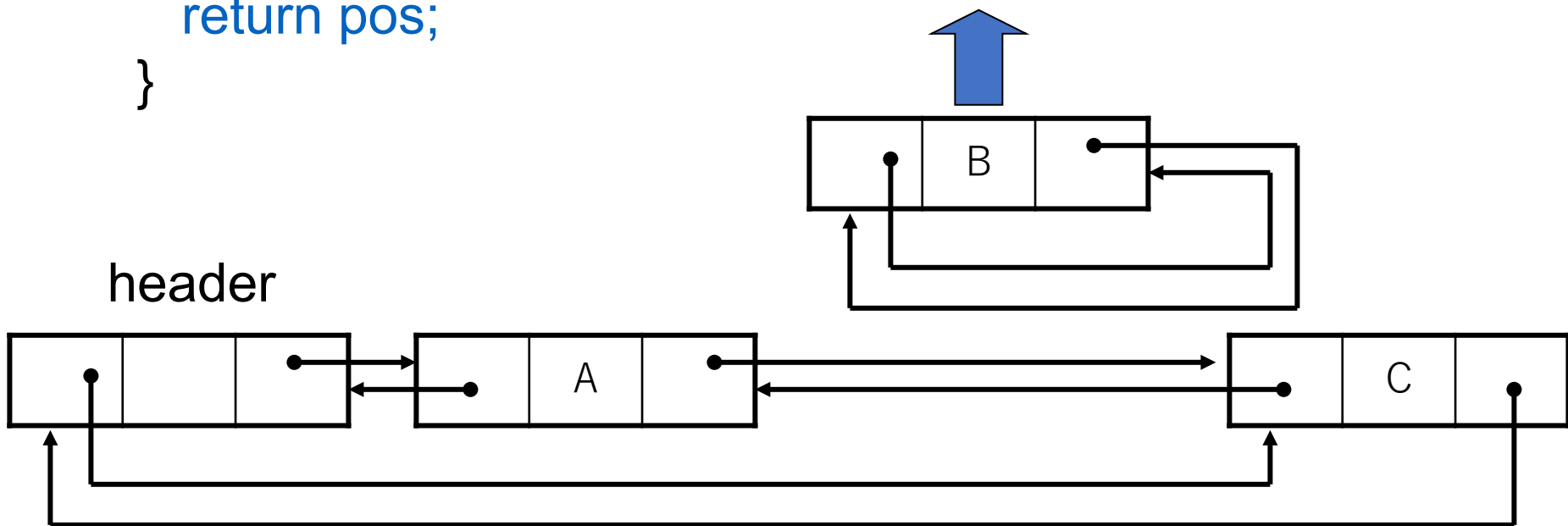


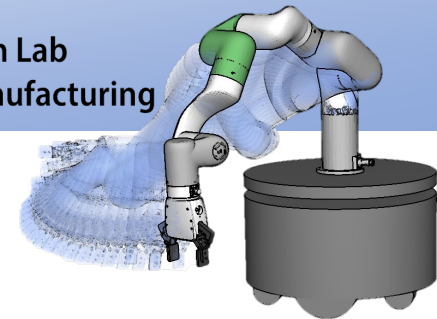


指定された要素を削除する

```

struct CELL *erase( struct CELL *pos ) {
    pos->prev->next = pos->next;
    pos->next->prev = pos->prev;
    pos->prev = pos->next = pos;
    return pos;
}
  
```



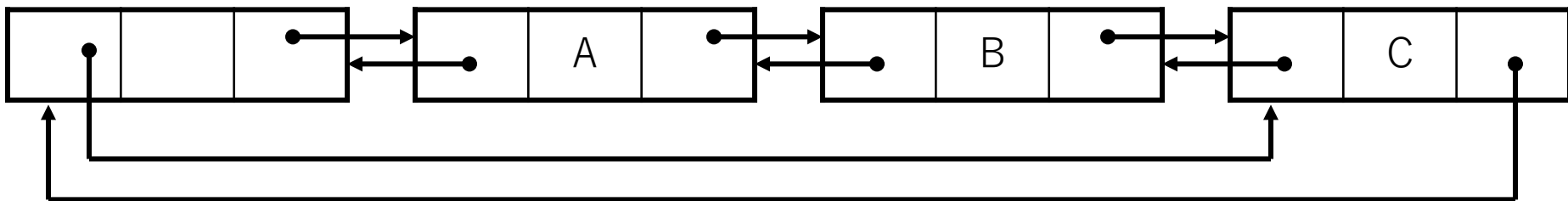


tail の実装

■ リストの最後尾を返す

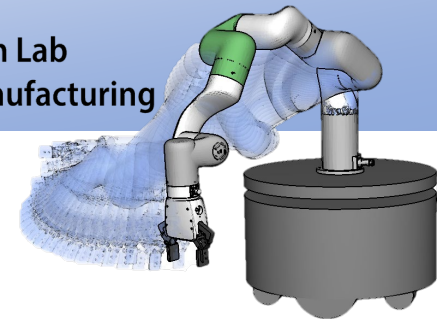
- 循環しているので header を見分ける必要がある.
- Header が分かれば, その前のセルへのポインタでよい.

header

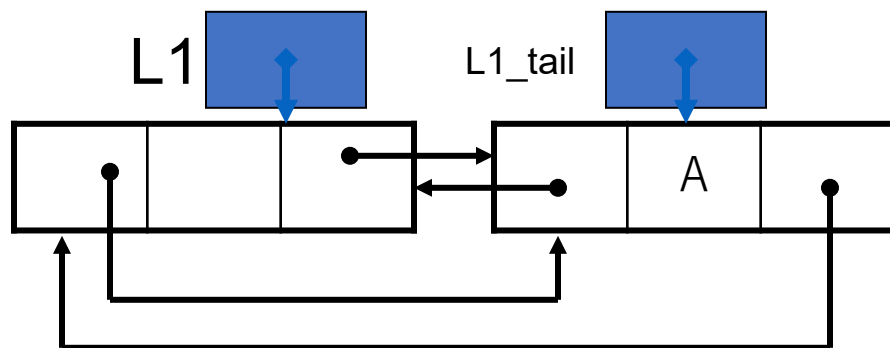


```

struct CELL *tail( struct CELL *header ){
    return header->prev; /* 事前条件として header は
                           リストのヘッダであること */
}
  
```

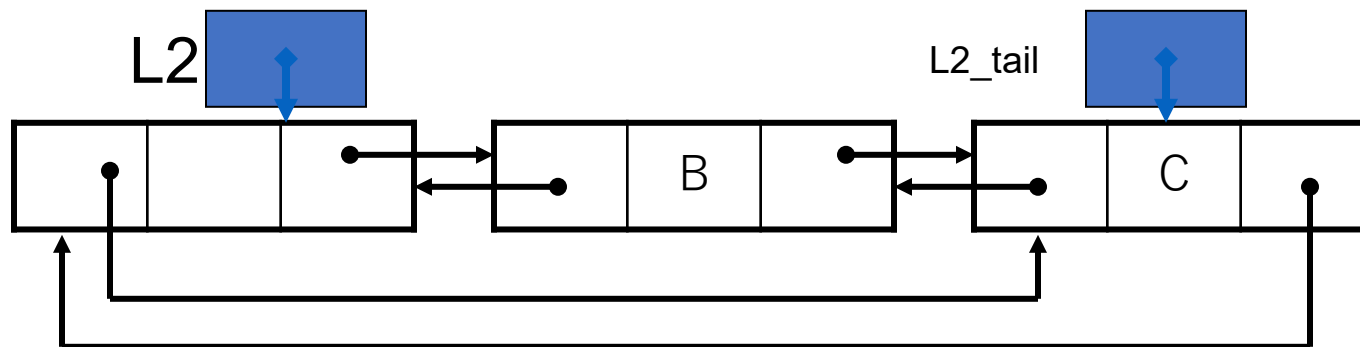


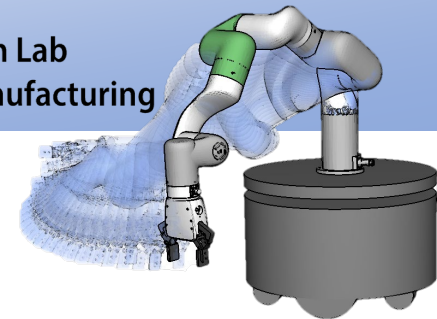
リストの連結(concatenate)



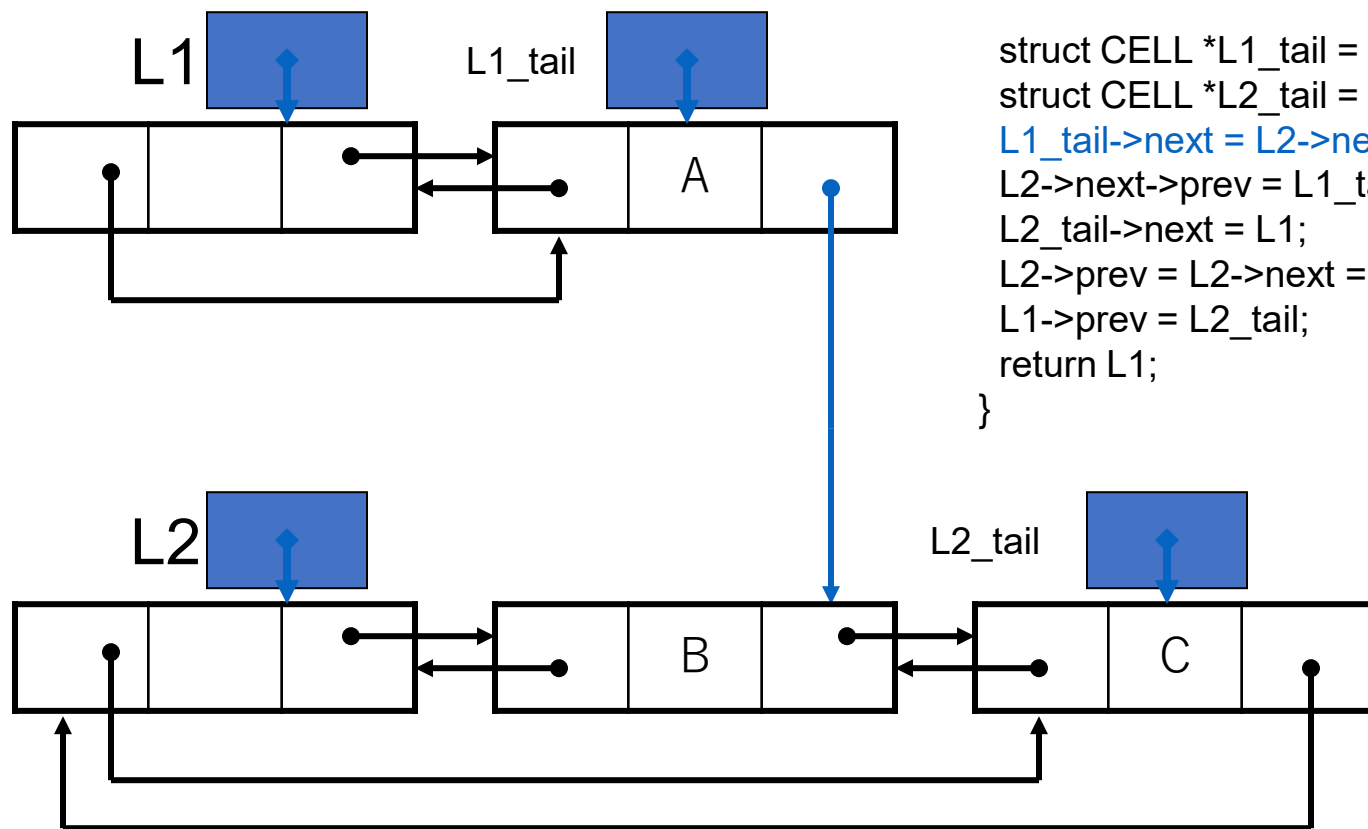
```

struct CELL *concatenate(struct CELL* L1,
                        struct CELL* L2){
    struct CELL *L1_tail = L1->prev;
    struct CELL *L2_tail = L2->prev;
    L1_tail->next = L2->next;
    L2->next->prev = L1_tail;
    L2_tail->next = L1;
    L2->prev = L2->next = L2;
    L1->prev = L2_tail;
    return L1;
}
  
```



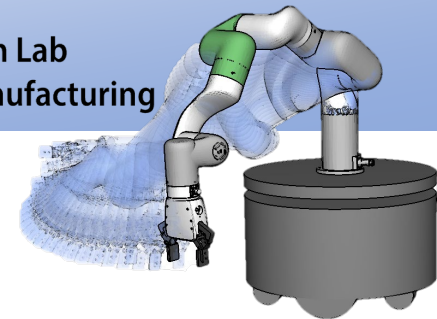


リストの連結(concatenate)

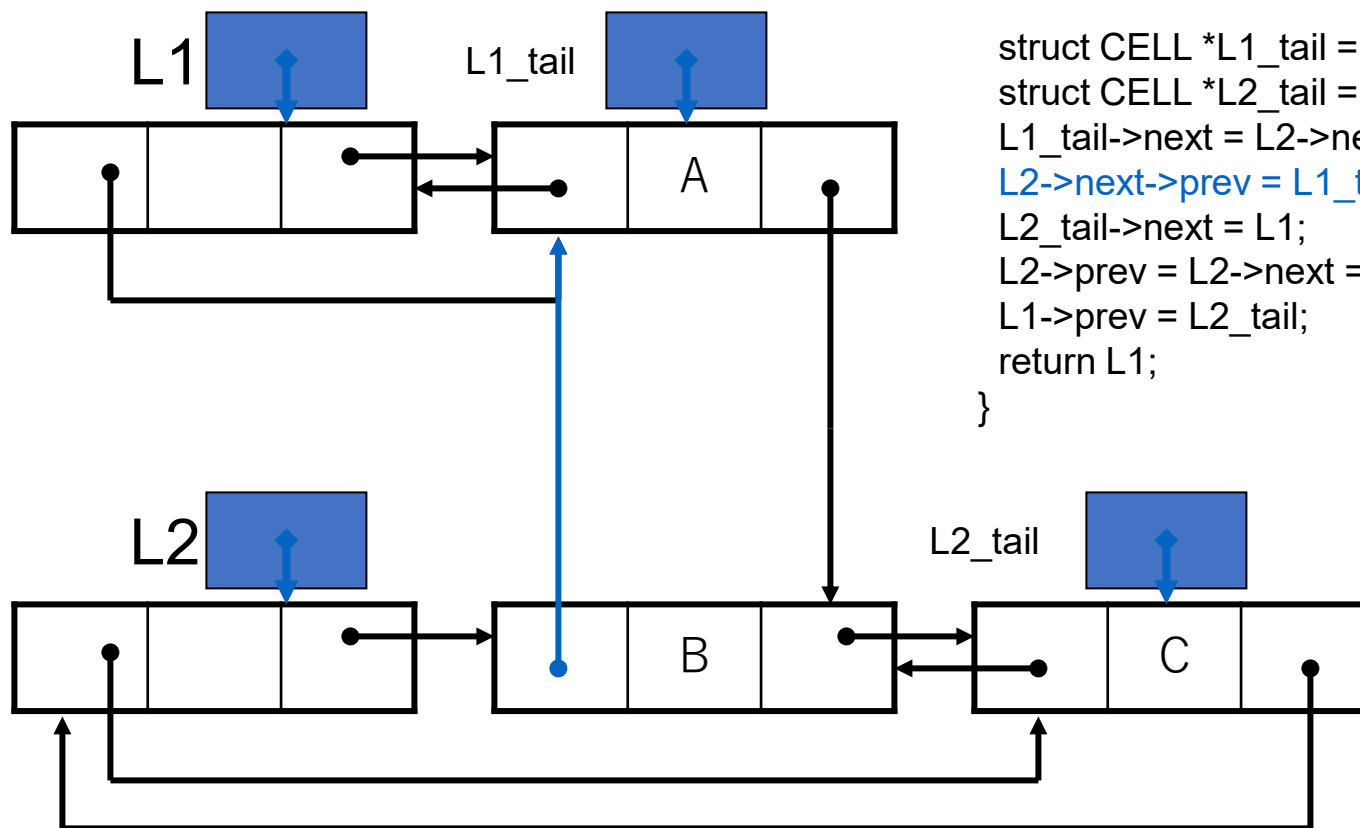


```

struct CELL *concatenate(struct CELL* L1,
                        struct CELL* L2){
    struct CELL *L1_tail = L1->prev;
    struct CELL *L2_tail = L2->prev;
    L1_tail->next = L2->next;
    L2->next->prev = L1_tail;
    L2_tail->next = L1;
    L2->prev = L2->next = L2;
    L1->prev = L2_tail;
    return L1;
}
  
```

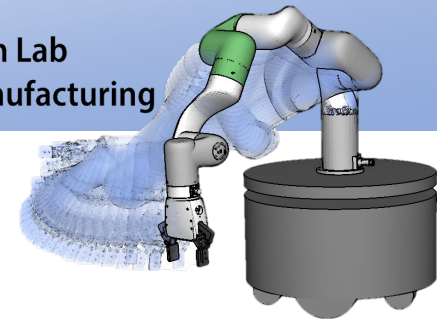


リストの連結(concatenate)



```

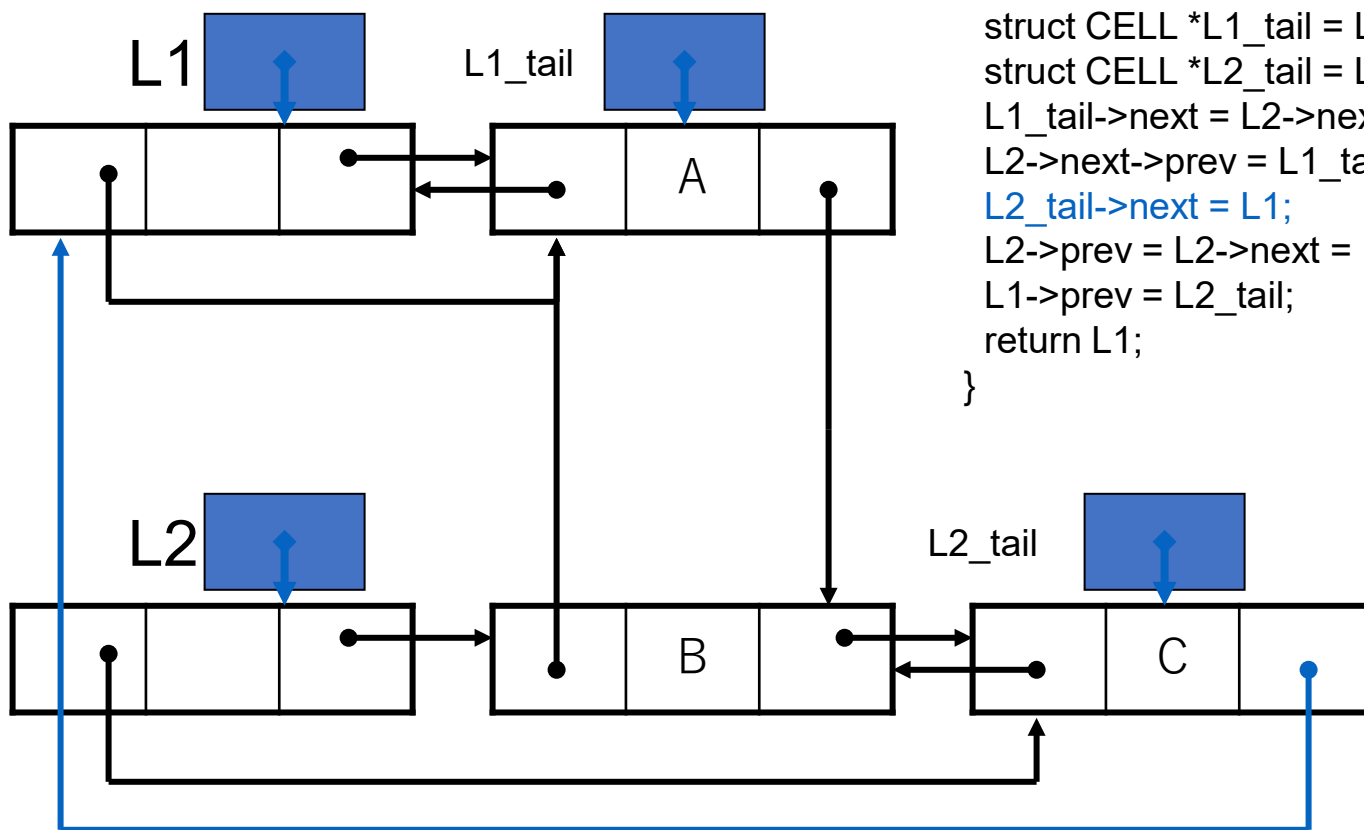
struct CELL *concatenate(struct CELL * L1,
                        struct CELL * L2){
    struct CELL *L1_tail = L1->prev;
    struct CELL *L2_tail = L2->prev;
    L1_tail->next = L2->next;
    L2->next->prev = L1_tail;
    L2_tail->next = L1;
    L2->prev = L2->next = L2;
    L1->prev = L2_tail;
    return L1;
}
  
```

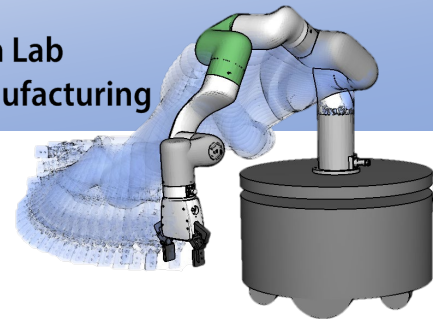


リストの連結(concatenate)

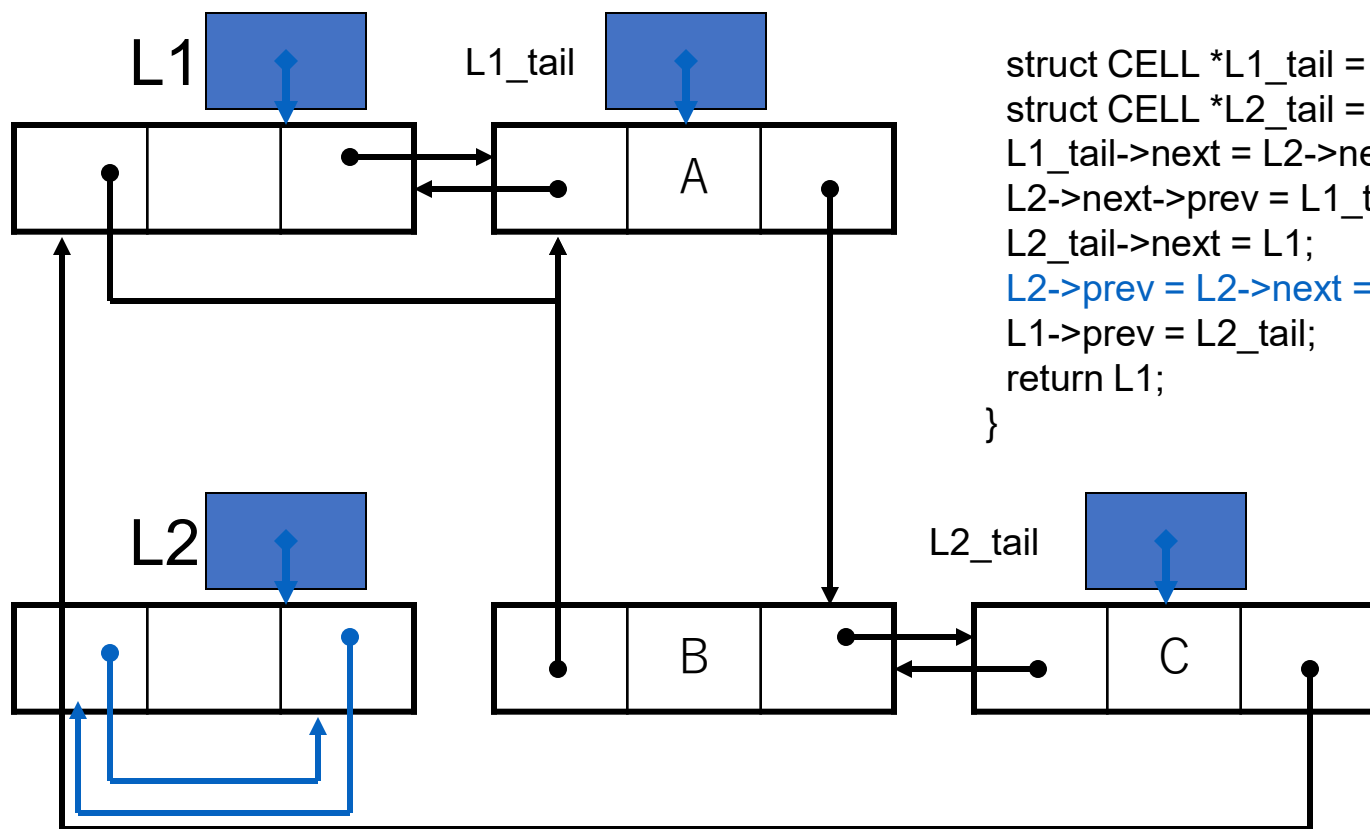
```

struct CELL *concatenate(struct CELL* L1,
                        struct CELL* L2){
    struct CELL *L1_tail = L1->prev;
    struct CELL *L2_tail = L2->prev;
    L1_tail->next = L2->next;
    L2->next->prev = L1_tail;
    L2_tail->next = L1;
    L2->prev = L2->next = L2;
    L1->prev = L2_tail;
    return L1;
}
  
```



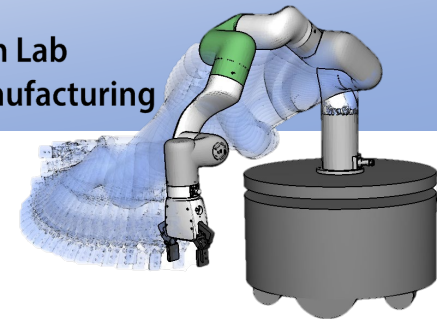


リストの連結(concatenate)

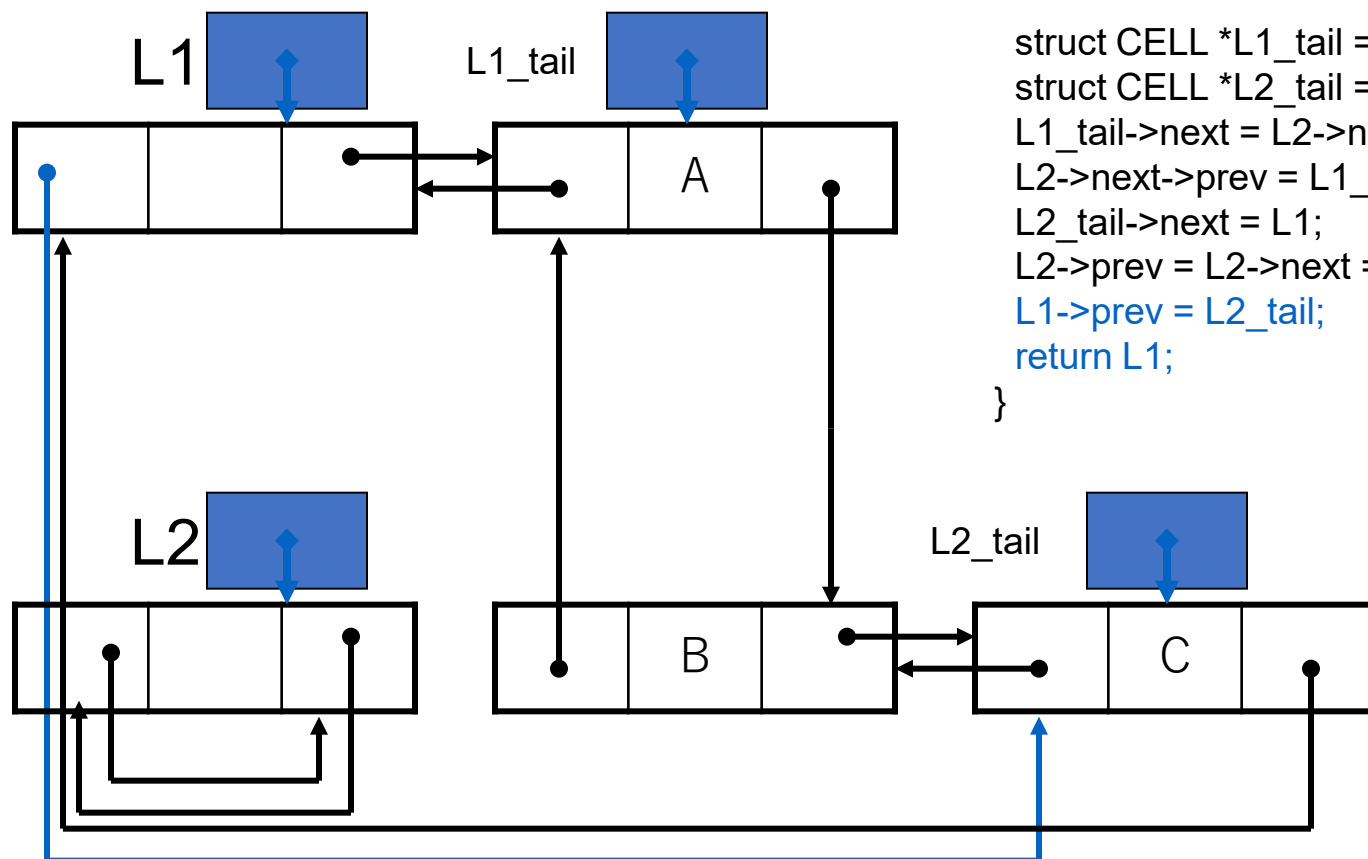


```

struct CELL *concatenate(struct CELL * L1,
                        struct CELL * L2){
    struct CELL *L1_tail = L1->prev;
    struct CELL *L2_tail = L2->prev;
    L1_tail->next = L2->next;
    L2->next->prev = L1_tail;
    L2_tail->next = L1;
    L2->prev = L2->next = L2;
    L1->prev = L2_tail;
    return L1;
}
  
```

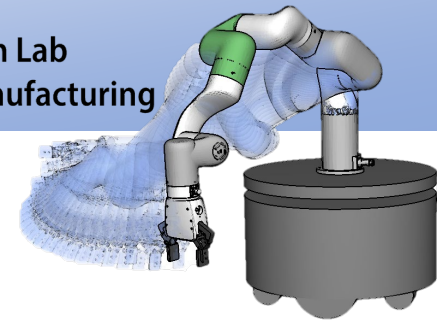


リストの連結(concatenate)

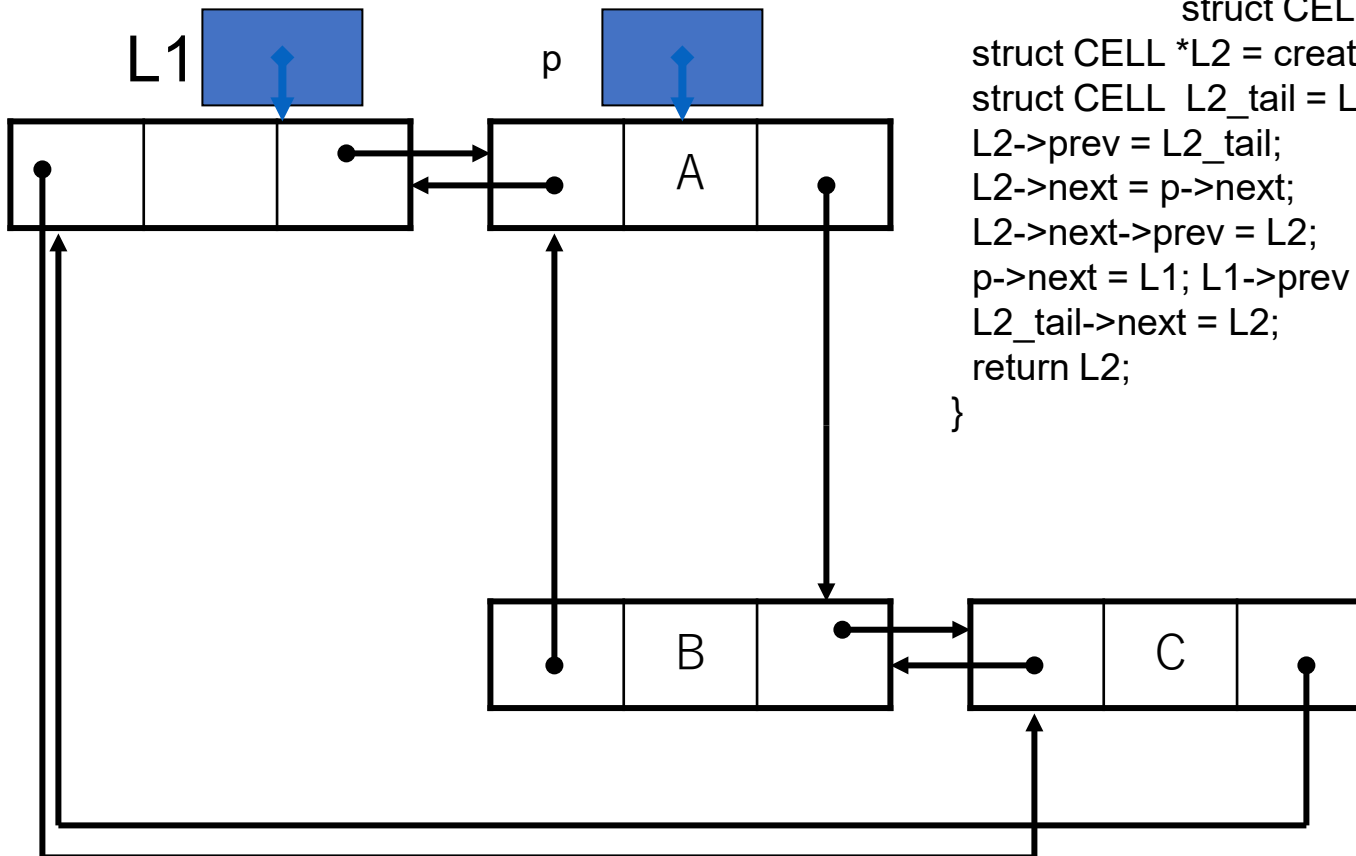


```

struct CELL *concatenate(struct CELL* L1,
                        struct CELL* L2){
    struct CELL *L1_tail = L1->prev;
    struct CELL *L2_tail = L2->prev;
    L1_tail->next = L2->next;
    L2->next->prev = L1_tail;
    L2_tail->next = L1;
    L2->prev = L2->next = L2;
    L1->prev = L2_tail;
    return L1;
}
  
```



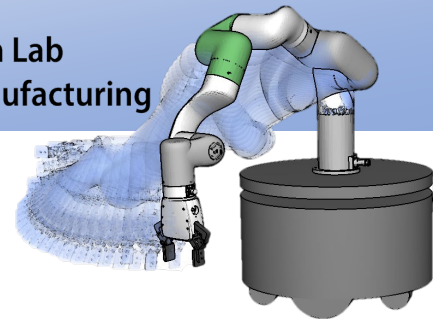
リストの分断(split)



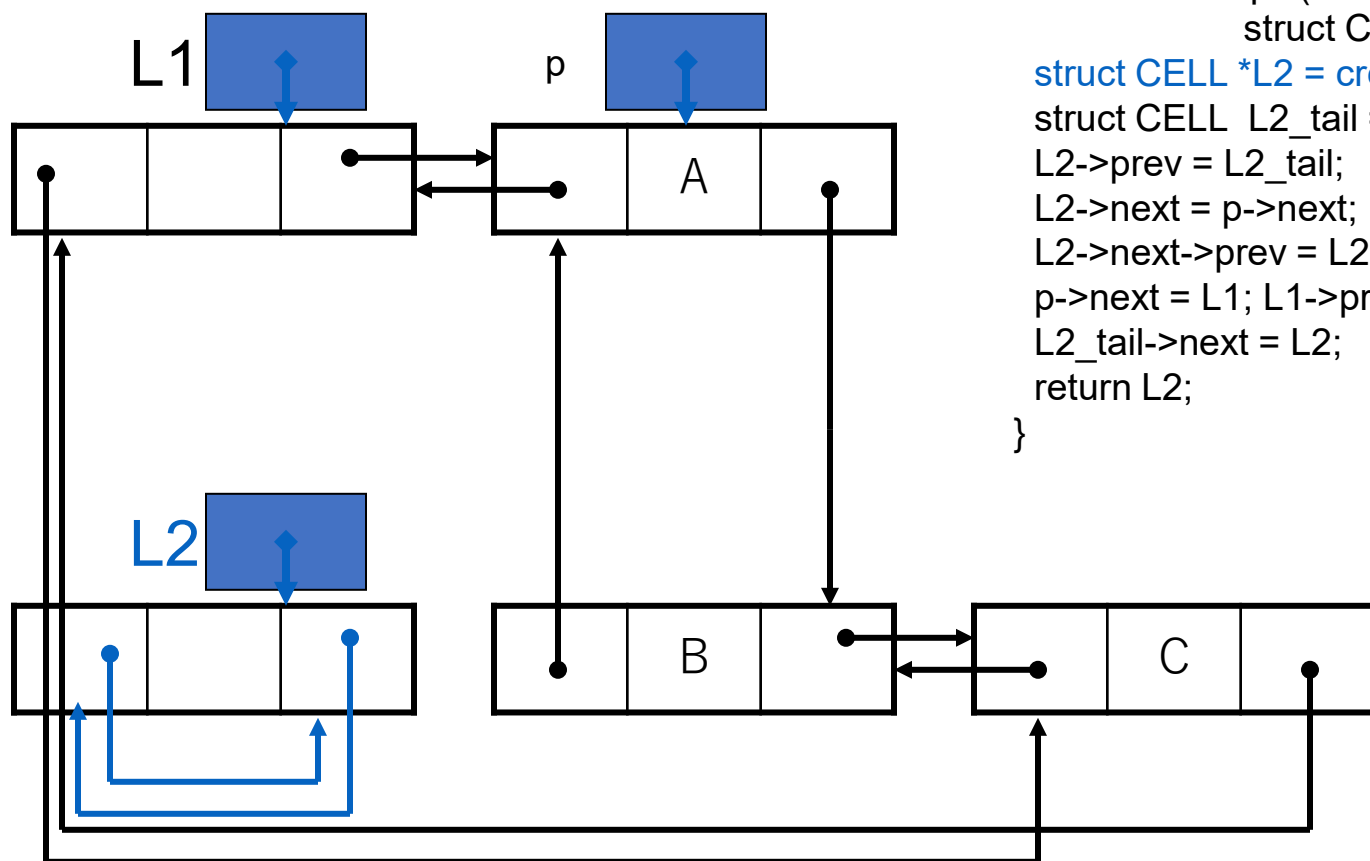
```

struct CELL *split(struct CELL *L1,
                  struct CELL *p) {
    struct CELL *L2 = create( L1->data );
    struct CELL L2_tail = L1->prev;
    L2->prev = L2_tail;
    L2->next = p->next;
    L2->next->prev = L2;
    p->next = L1; L1->prev = p;
    L2_tail->next = L2;
    return L2;
}
  
```



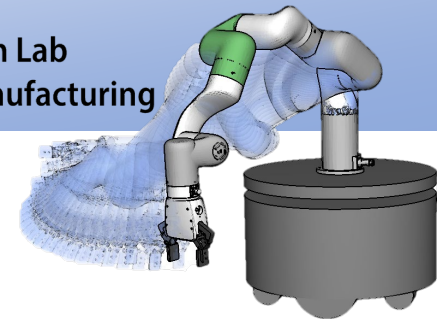


リストの分断(split)

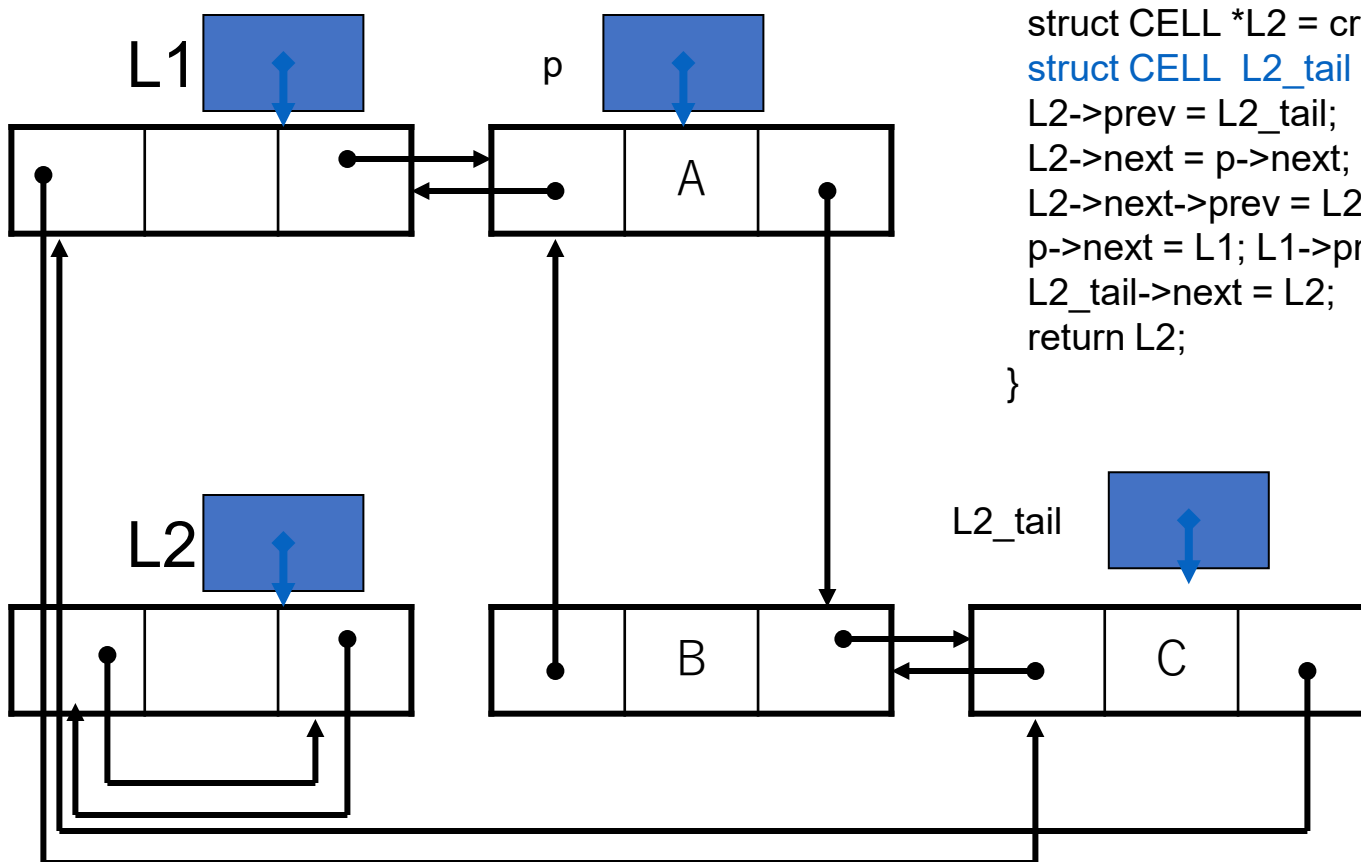


```

struct CELL *split(struct CELL *L1,
                  struct CELL *p) {
    struct CELL *L2 = create( L1->data );
    struct CELL L2_tail = L1->prev;
    L2->prev = L2_tail;
    L2->next = p->next;
    L2->next->prev = L2;
    p->next = L1; L1->prev = p;
    L2_tail->next = L2;
    return L2;
}
  
```

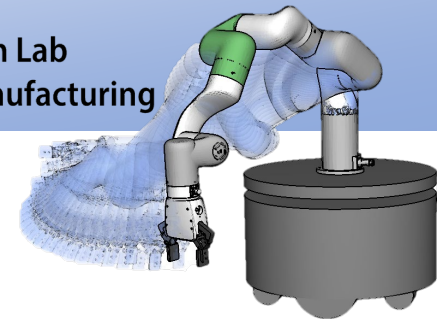


リストの分断(split)

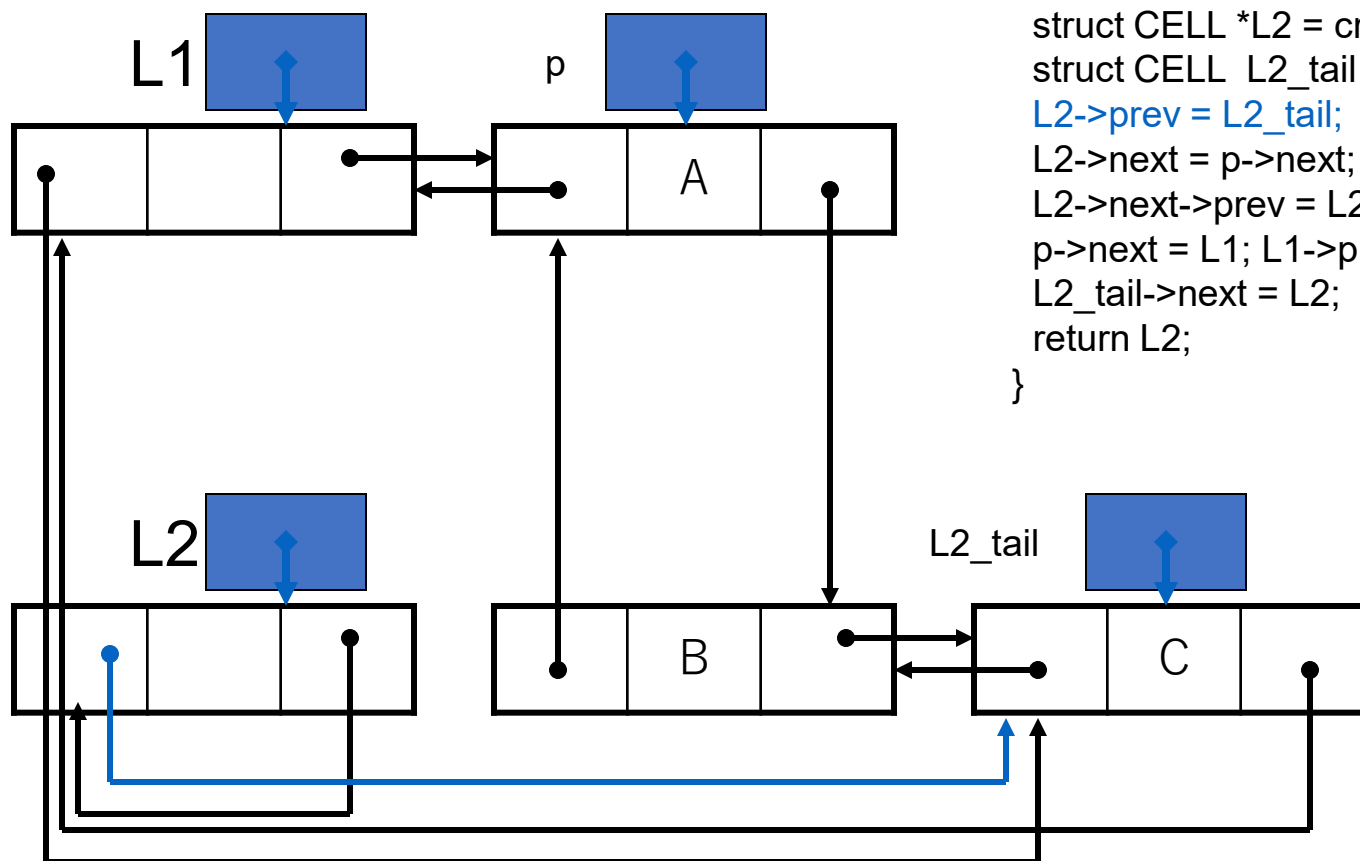


```

struct CELL *split(struct CELL *L1,
                  struct CELL *p) {
    struct CELL *L2 = create( L->data );
    struct CELL L2_tail = L1->prev;
    L2->prev = L2_tail;
    L2->next = p->next;
    L2->next->prev = L2;
    p->next = L1; L1->prev = p;
    L2_tail->next = L2;
    return L2;
}
  
```

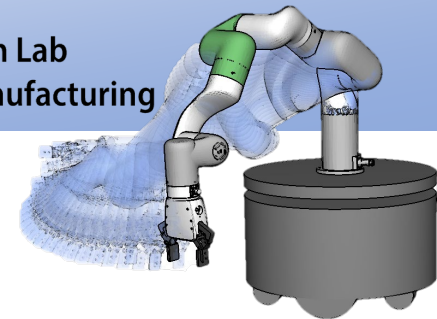


リストの分断(split)

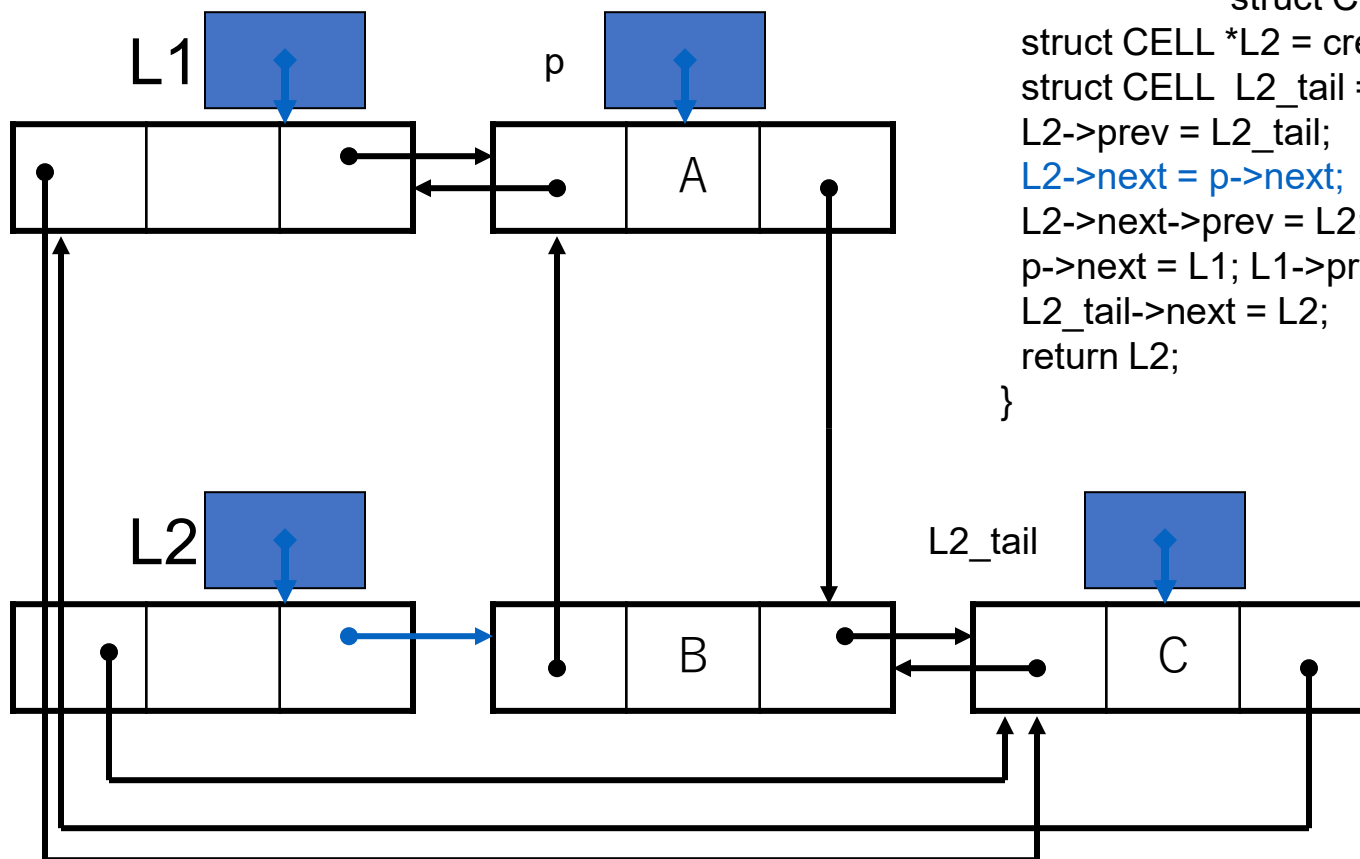


```

struct CELL *split(struct CELL *L1,
                  struct CELL *p) {
    struct CELL *L2 = create( L->data );
    struct CELL L2_tail = L1->prev;
    L2->prev = L2_tail;
    L2->next = p->next;
    L2->next->prev = L2;
    p->next = L1; L1->prev = p;
    L2_tail->next = L2;
    return L2;
}
  
```

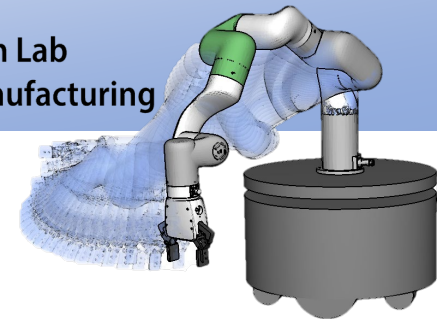


リストの分断(split)

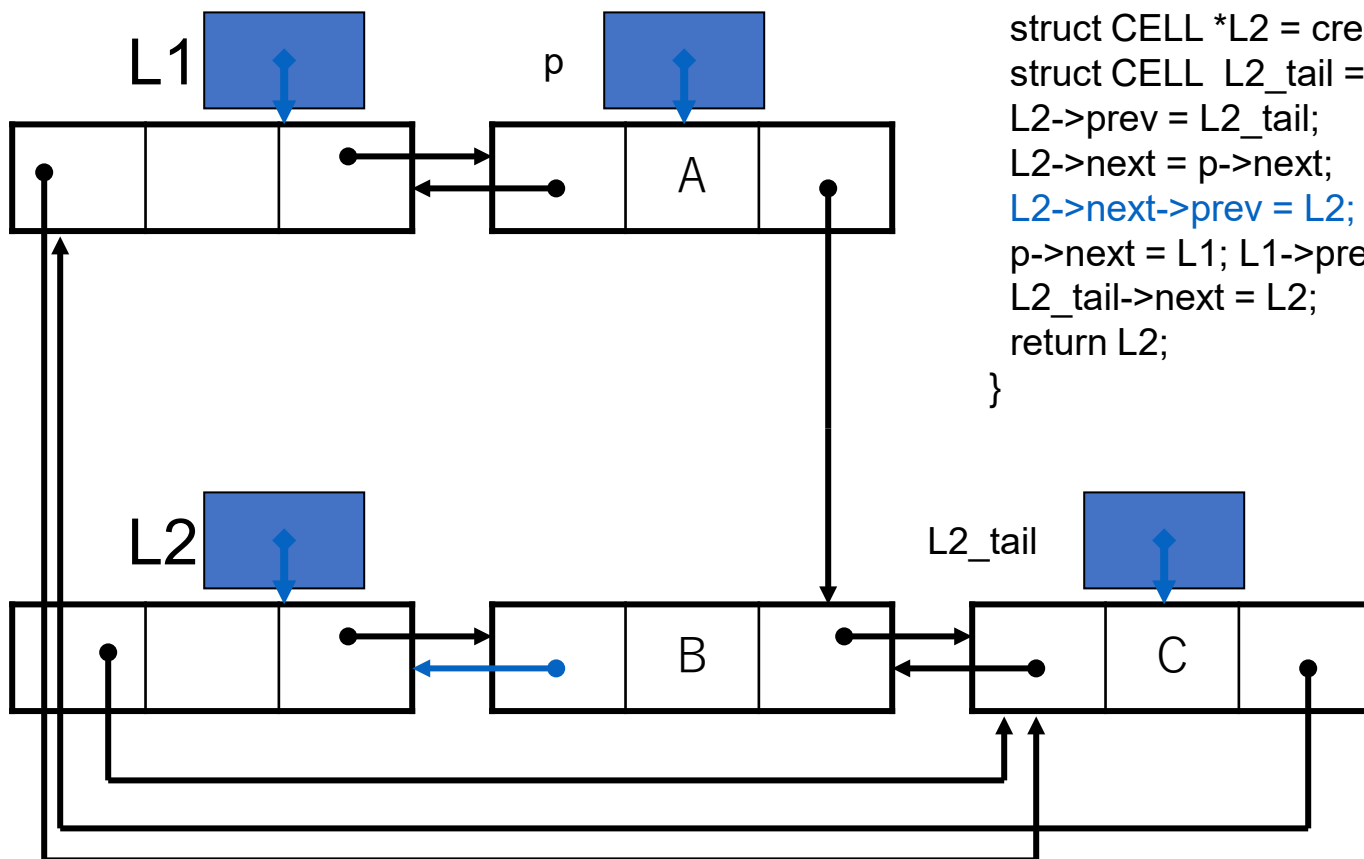


```

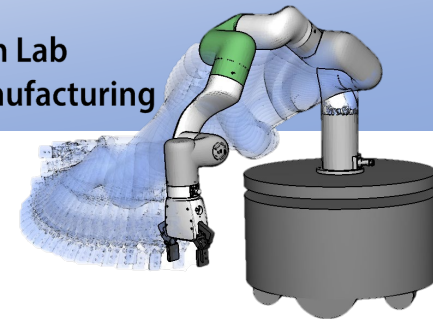
struct CELL *split(struct CELL *L1,
                  struct CELL *p) {
    struct CELL *L2 = create( L->data );
    struct CELL L2_tail = L1->prev;
    L2->prev = L2_tail;
    L2->next = p->next;
    L2->next->prev = L2;
    p->next = L1; L1->prev = p;
    L2_tail->next = L2;
    return L2;
}
  
```



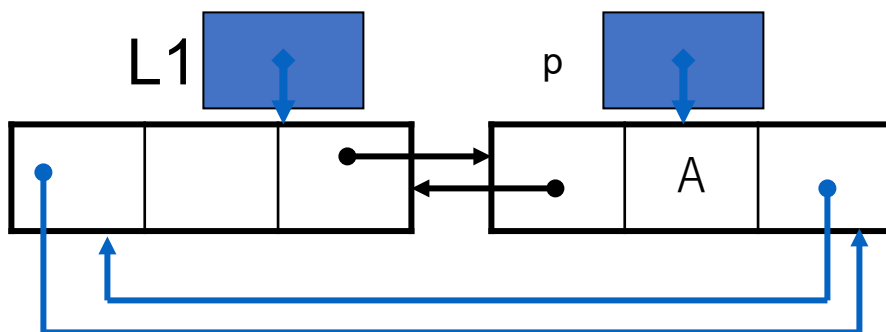
リストの分断(split)



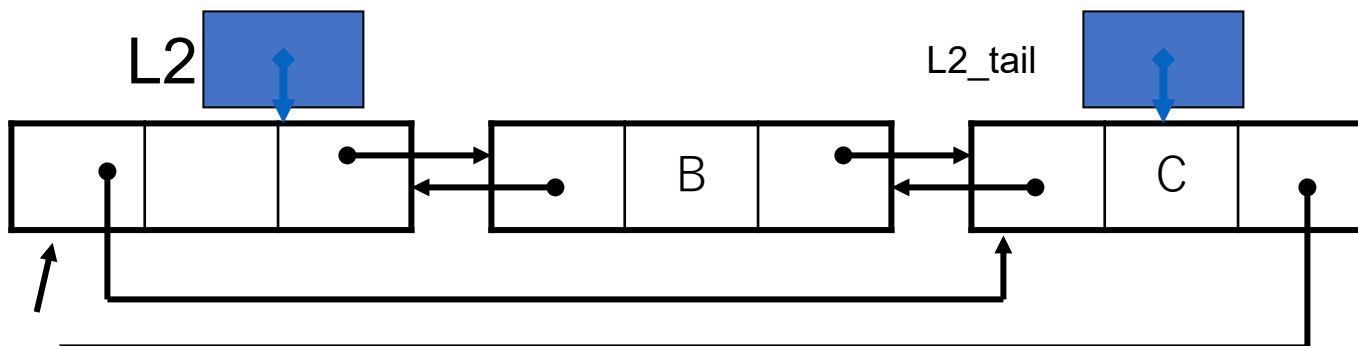
```
struct CELL *split(struct CELL *L1,
                  struct CELL *p) {
    struct CELL *L2 = create( L->data );
    struct CELL L2_tail = L1->prev;
    L2->prev = L2_tail;
    L2->next = p->next;
    L2->next->prev = L2;
    p->next = L1; L1->prev = p;
    L2_tail->next = L2;
    return L2;
}
```

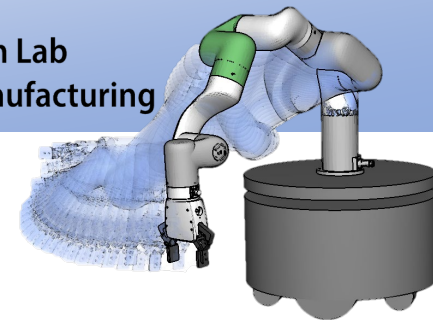


リストの分断(split)

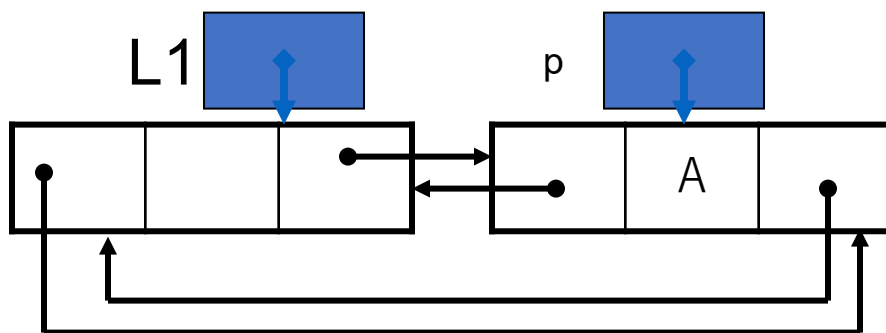


```
struct CELL *split(struct CELL *L1,
                  struct CELL *p) {
    struct CELL *L2 = create( L->data );
    struct CELL L2_tail = L1->prev;
    L2->prev = L2_tail;
    L2->next = p->next;
    L2->next->prev = L2;
    p->next = L1; L1->prev = p;
    L2_tail->next = L2;
    return L2;;
}
```

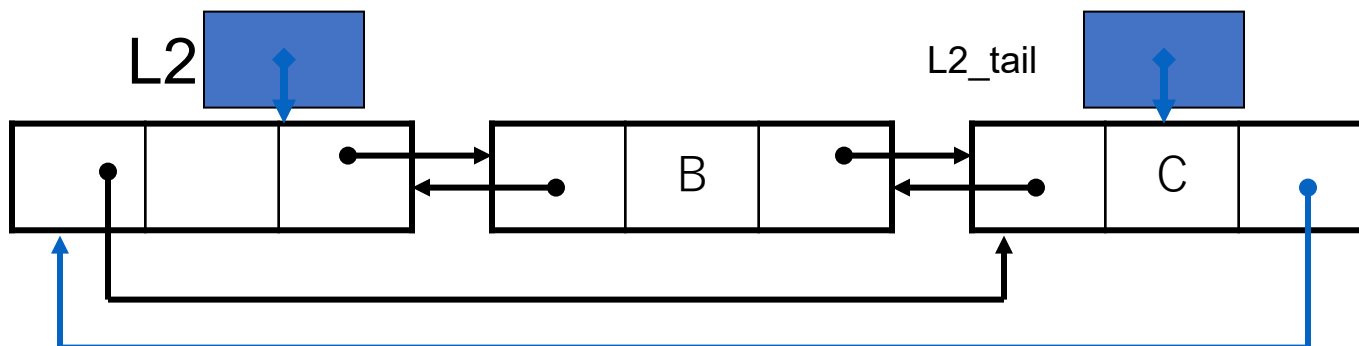


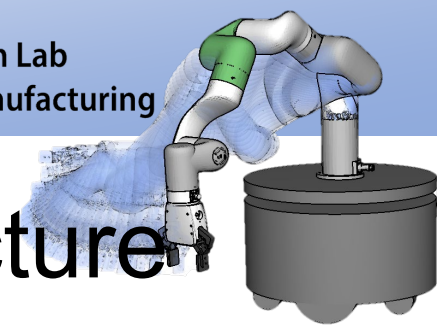


リストの分断(split)



```
struct CELL *split(struct CELL *L1,
                  struct CELL *p) {
    struct CELL *L2 = create( L->data );
    struct CELL L2_tail = L1->prev;
    L2->prev = L2_tail;
    L2->next = p->next;
    L2->next->prev = L2;
    p->next = L1; L1->prev = p;
    L2_tail->next = L2;
    return L2;
}
```



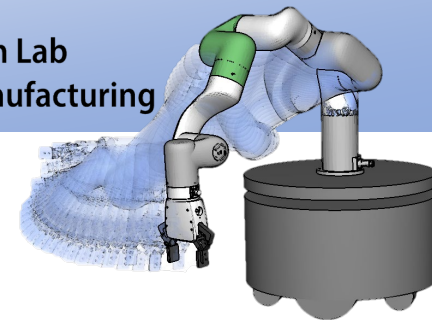


多重リスト構造 multi-link structure

- セルに2つ以上のポインタを持たせ, 同時に複数のリストに所属できるようにしたデータ構造
- 例) 大学での学生の成績を管理する表
 - 2次元配列でも表現可能

```
short credit[MAX_STUDENT][MAX_SUBJECT];
```

credit[x][y] 学生番号 x の学生の課目 y の成績
未受講 -1

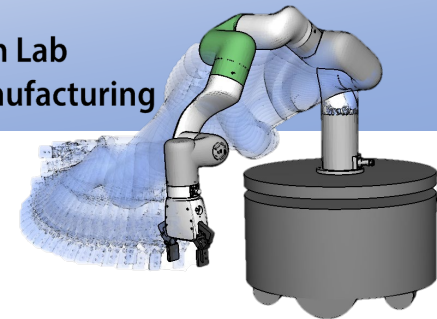


2次元配列を用いた場合

課目数

	0	1	2	3	4						
0	90		83								
1	72			90							
2		52	53								
3				25							
4					23						
				44	55						

学籍番号



例) 成績管理表

- 学生総数2,000人, 授業課目数400, 平均受講
課目数 10

平均履修者数 $\rightarrow 2000 \times 10 / 400 = 50$ 名

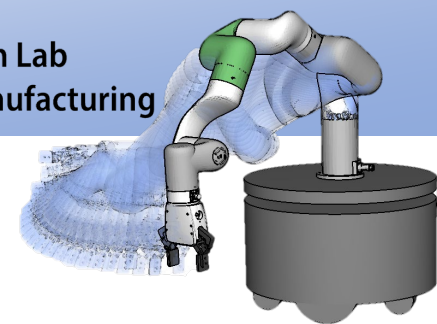
配列の大きさ $\rightarrow 2000 \times 400 \times \text{sizeof}(\text{short}) =$

データ量 $\rightarrow 2000 \times 10 \times \text{sizeof}(\text{short}) =$

配列の使用効率 $\rightarrow 1/40$ (無駄が多い)

sparse matrix

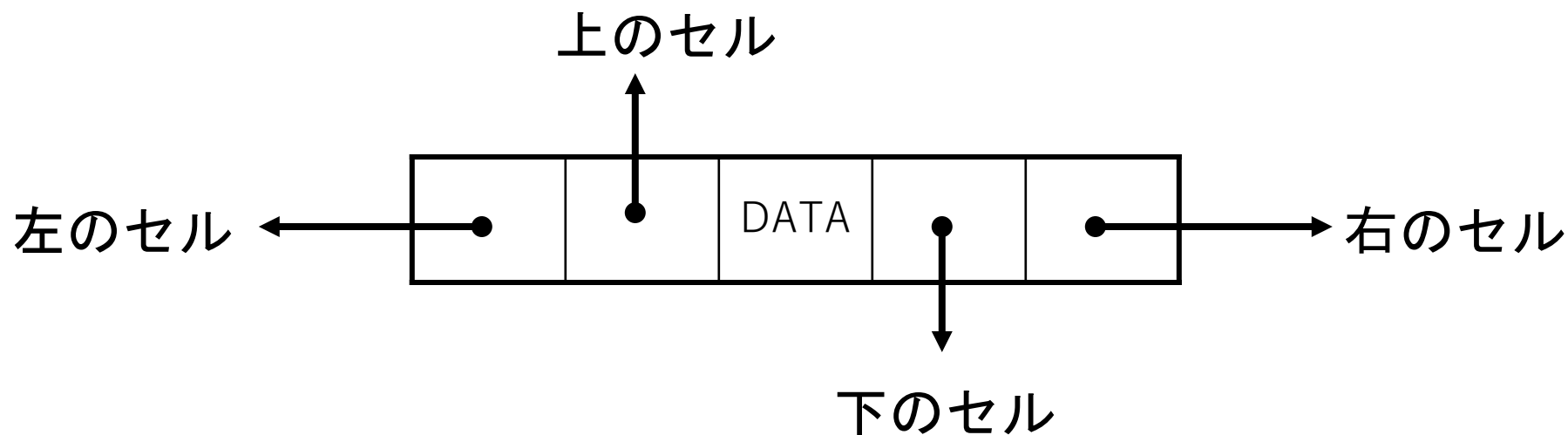
疎な配列を効率良く表現できる $\rightarrow$ 多重リスト構造

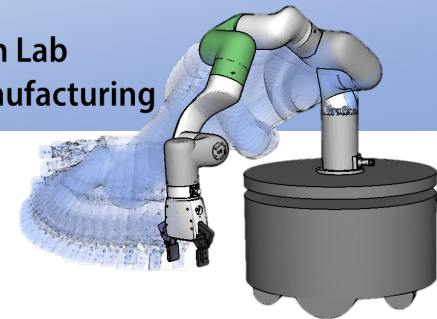


多重リスト構造

- 基本アイデア

- データのある要素をセルで表現し，それらをポインタでつなぐ





多重リスト（例：成績管理）

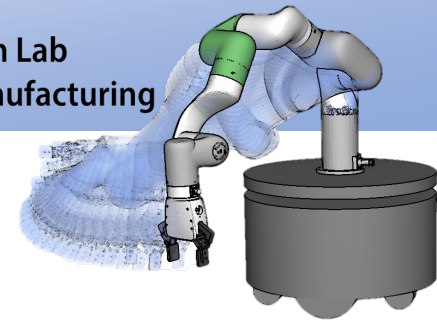
```
#define MAX_STUDENT 2000
#define MAX_SUBJECT 400

struct CREDIT {
    struct STUDENT *student; /* 学生先頭データ */
    struct SUBJECT *subject; /* 科目先頭データ */
    int    score;             /* 得点 */
    struct CREDIT *st_link;   /* 学生のリンク */
    struct CREDIT *su_link;   /* 科目のリンク */
};

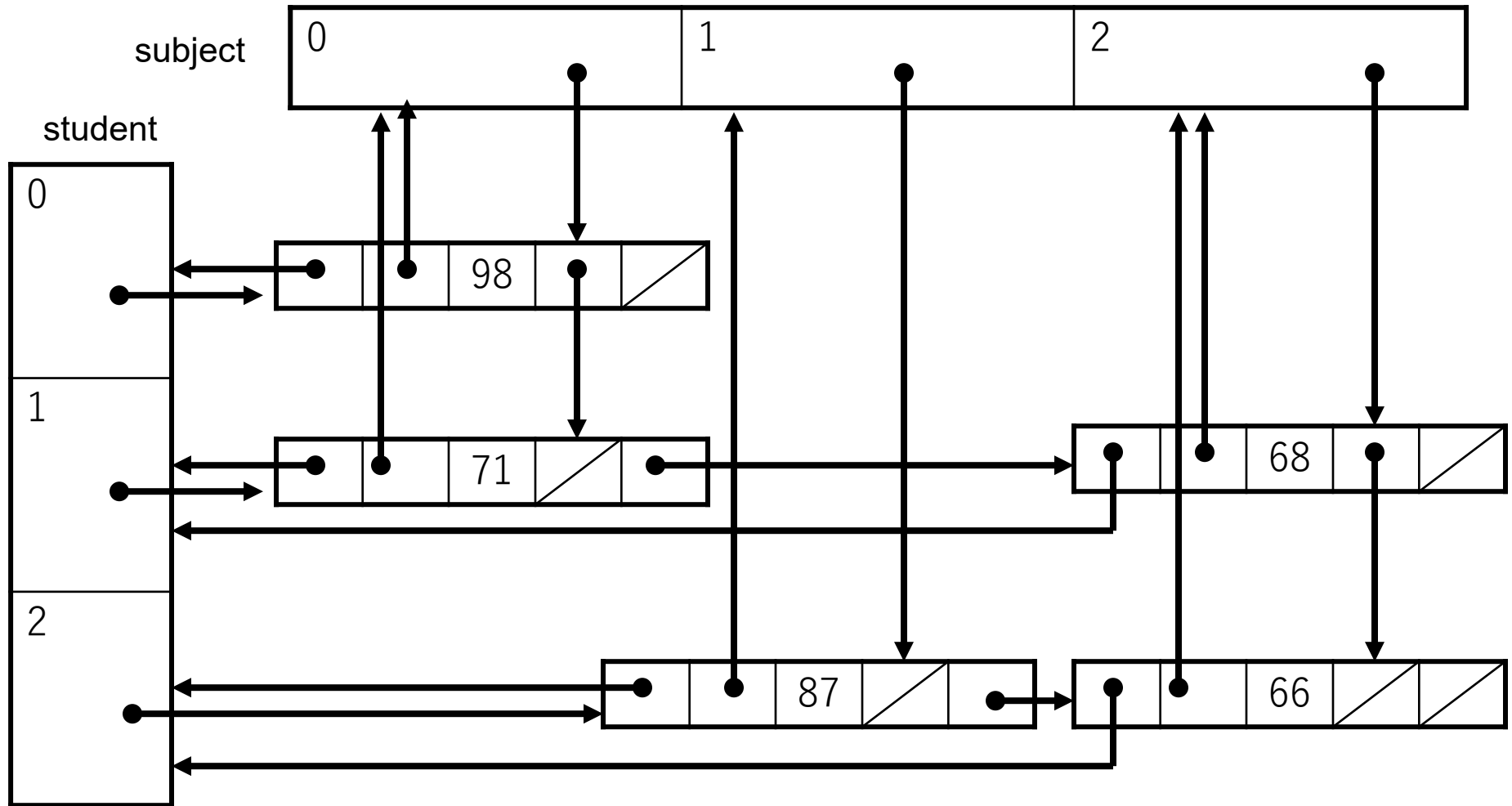
struct STUDENT {
    struct CREDIT *su_link; /* 履修科目 */
    char *name;
} Students[MAX_STUDENT];

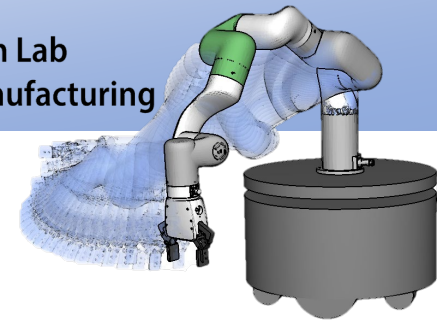
struct SUBJECT {
    struct CREDIT *st_link; /* 学生 */
    char *name;
} Subjects[MAX_SUBJECT];
```

student
subject
score
st_link
su_link

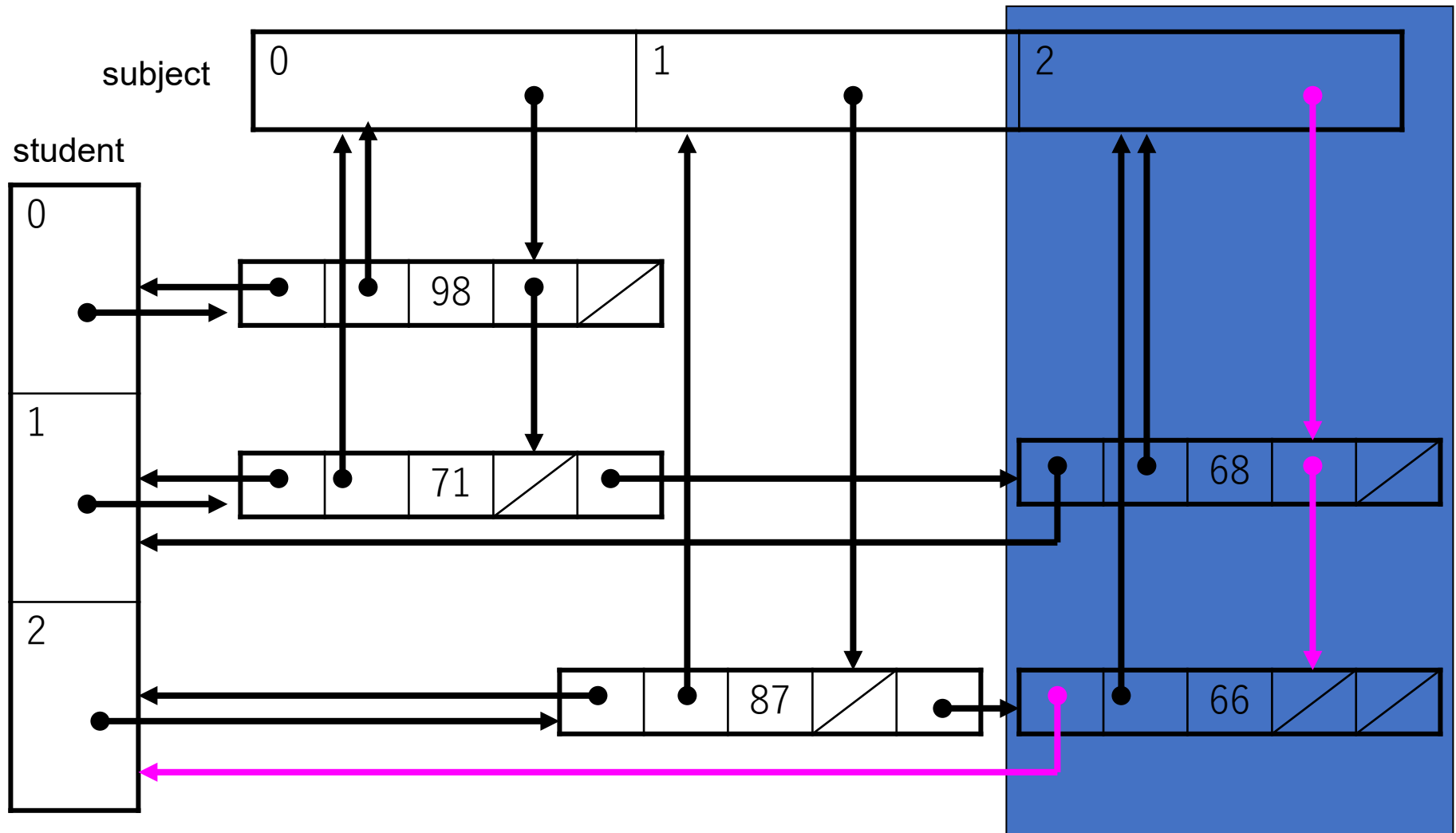


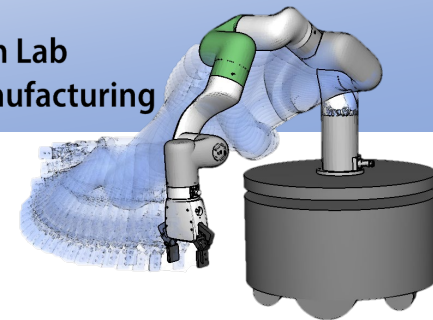
多重リスト（例：成績管理）



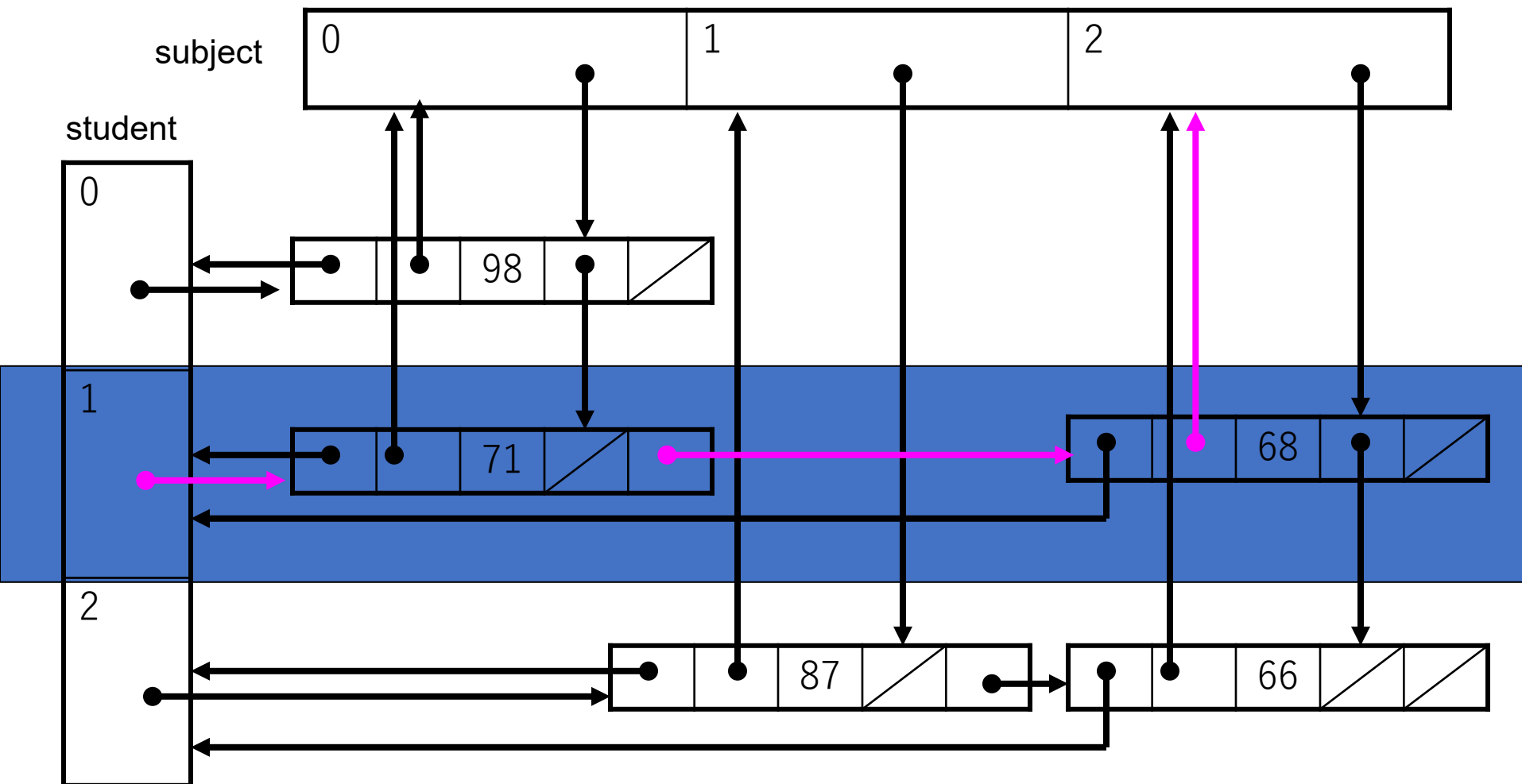


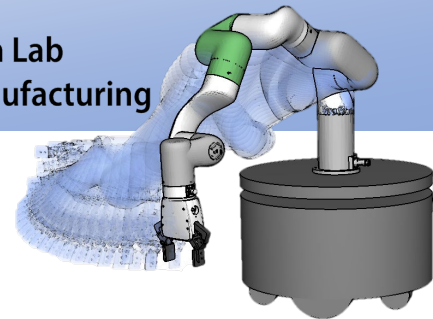
多重リスト（例：成績管理）





多重リスト（例：成績管理）

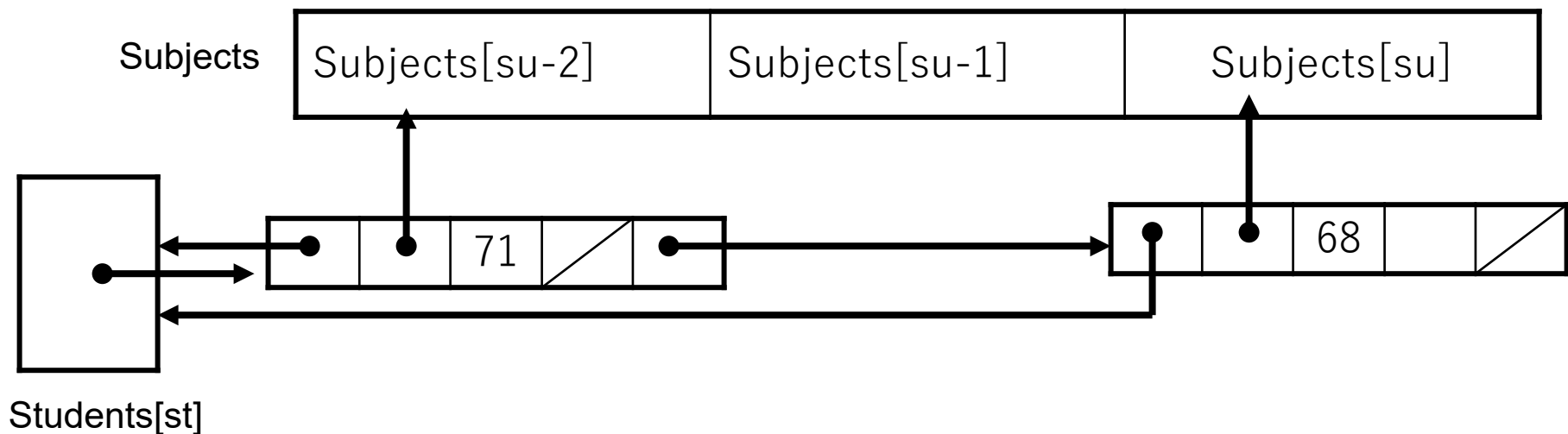


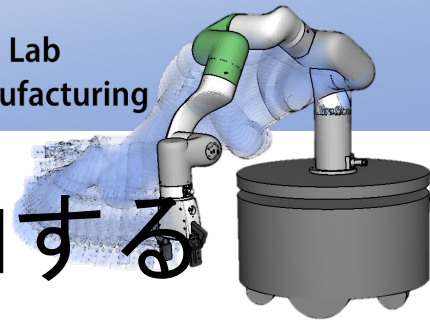


学生stの科目suの成績を得る

```

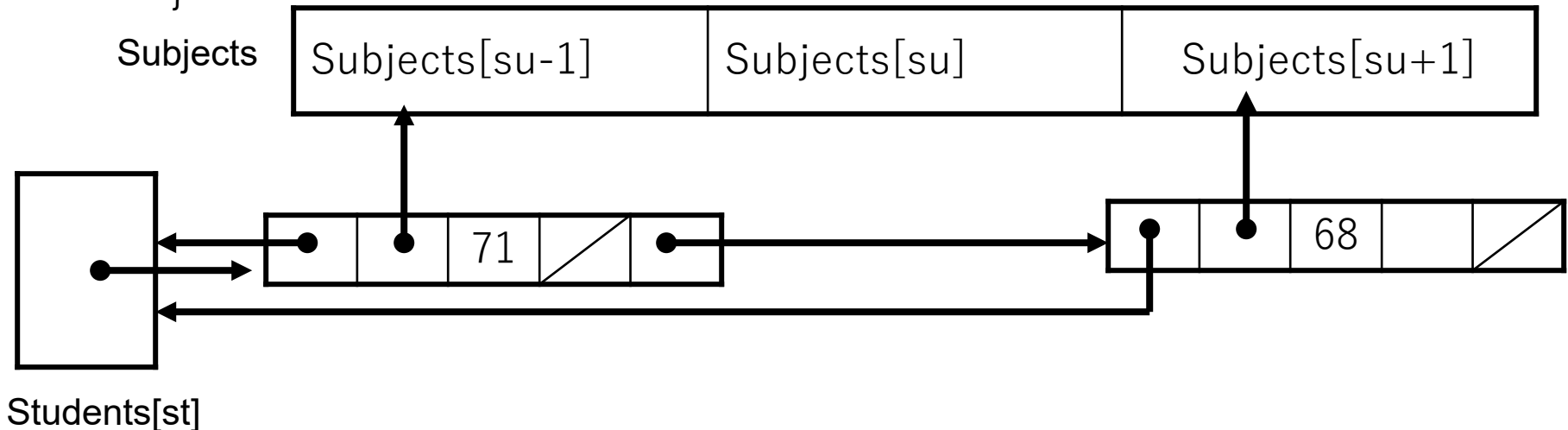
int get_score(int st,int su) {
    struct CREDIT *p;
    for (p = Students[st].su_link; p != NULL; p = p->su_link)
        if (p->subject == &Subjects[su])
            return p->score;
    return -1;
}
  
```

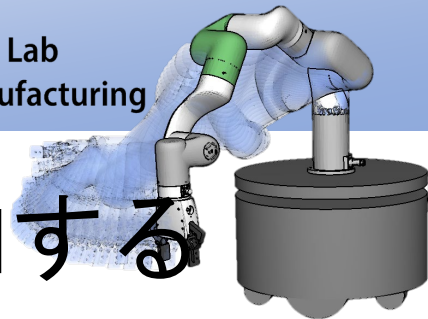




学生stの科目suの成績scを追加する

```
void add_credit(int st,int su,int sc) {
    struct CREDIT *p;
    if (get_score(st,su) == -1) {
        p = (struct CREDIT *)malloc(sizeof(struct CREDIT));
        p->student = &Students[st]; p->subject = &Subjects[su];
        p->score = sc;
        p->st_link = Subjects[su].st_link; Subjects[su].st_link = p;
        p->su_link = Students[st].su_link; Students[st].su_link = p;
    }
}
```

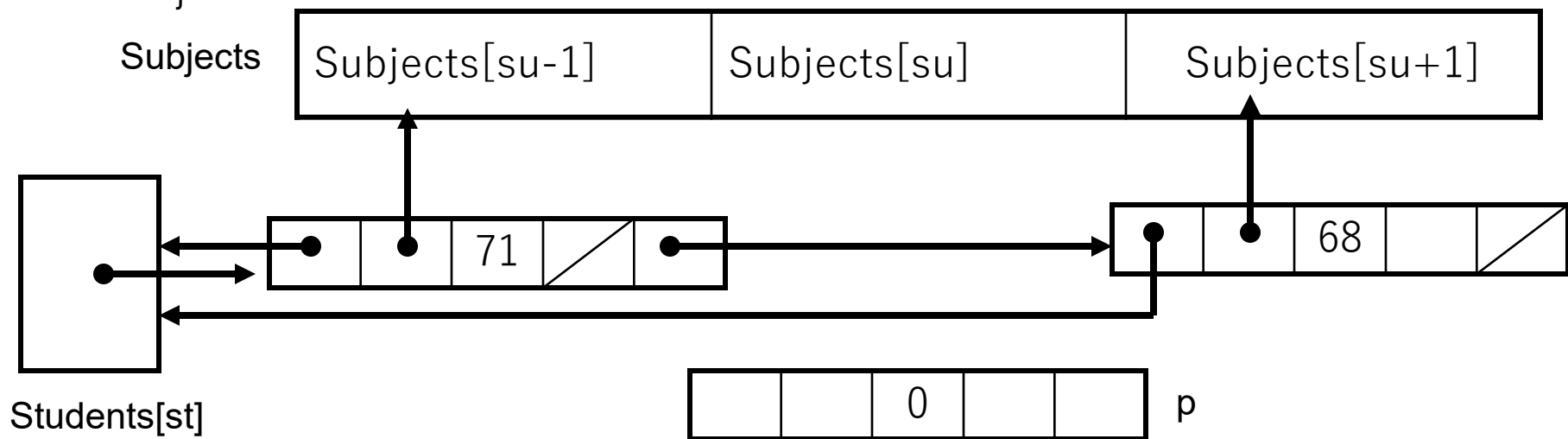


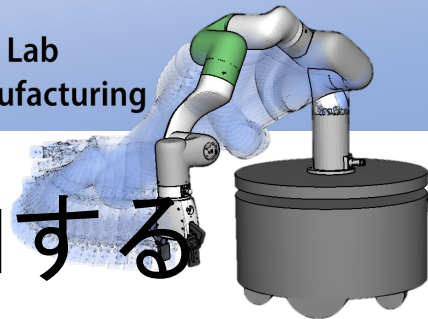


学生stの科目suの成績scを追加する

```

void add_credit(int st,int su,int sc) {
  struct CREDIT *p;
  if (get_score(st,su) == -1) {
    p = (struct CREDIT *)malloc(sizeof(struct CREDIT));
    p->student = &Students[st]; p->subject = &Subjects[su];
    p->score = sc;
    p->st_link = Subjects[su].st_link; Subjects[su].st_link = p;
    p->su_link = Students[st].su_link; Students[st].su_link = p;
  }
}
  
```

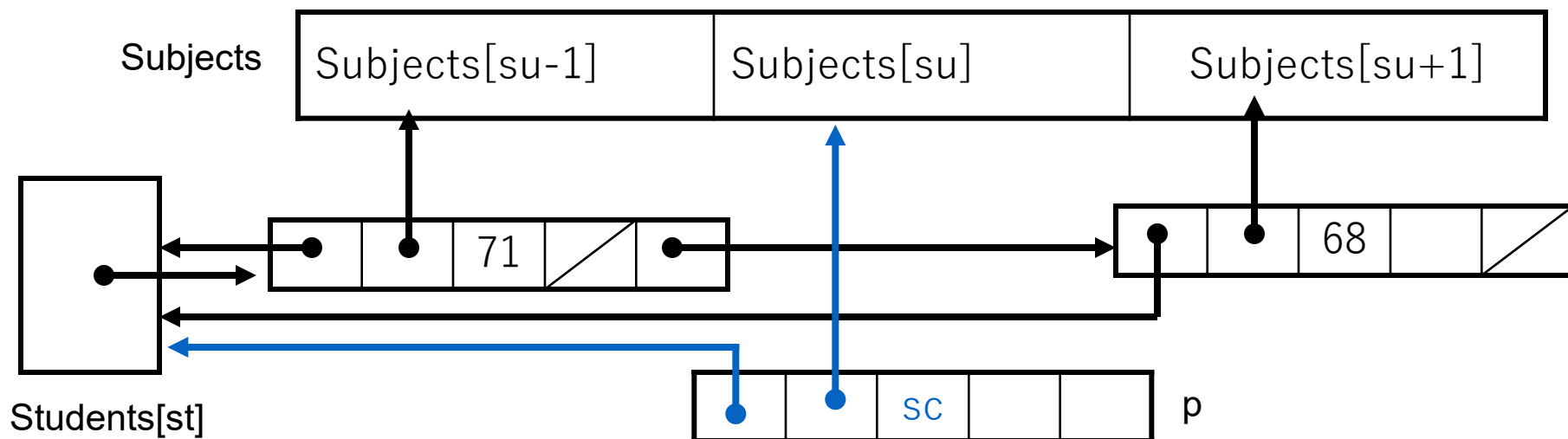


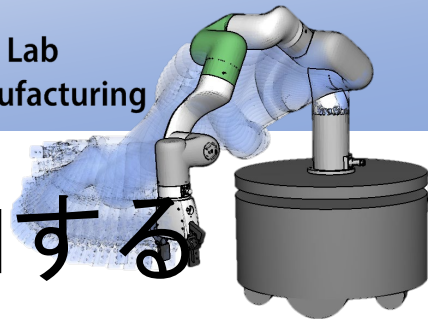


学生stの科目suの成績scを追加する

```

void add_credit(int st,int su,int sc) {
    struct CREDIT *p;
    if (get_score(st,su) == -1) {
        p = (struct CREDIT *)malloc(sizeof(struct CREDIT));
        p->student = &Students[st]; p->subject = &Subjects[su];
        p->score = sc;
        p->st_link = Subjects[su].st_link; Subjects[su].st_link = p;
        p->su_link = Students[st].su_link; Students[st].su_link = p;
    }
}
    
```

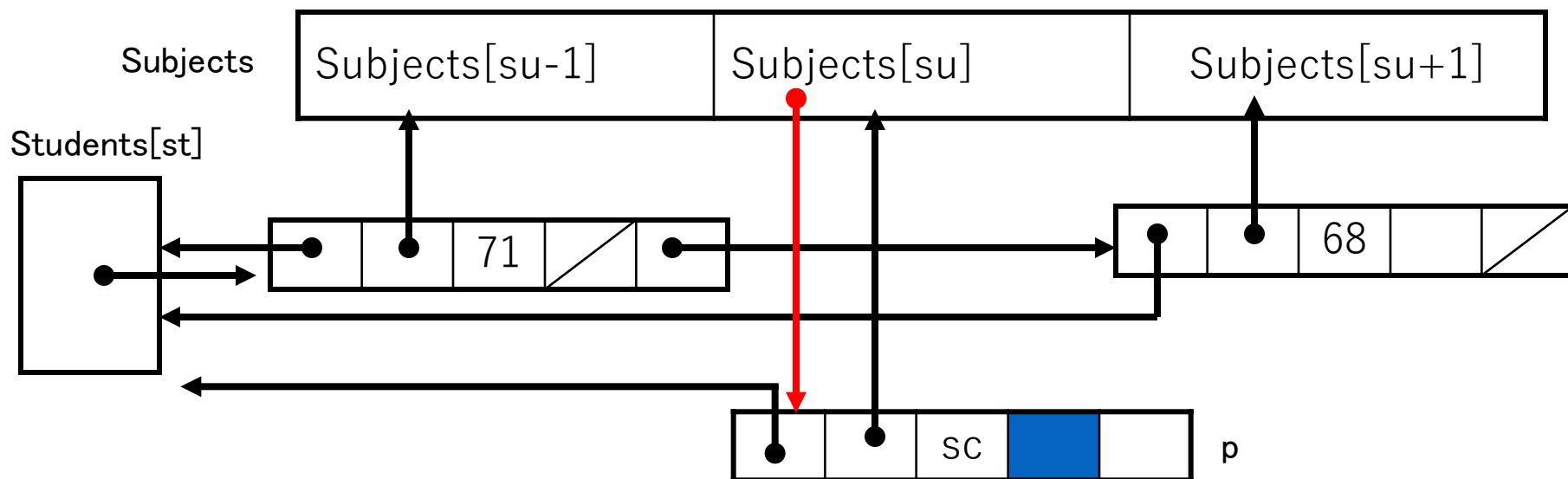


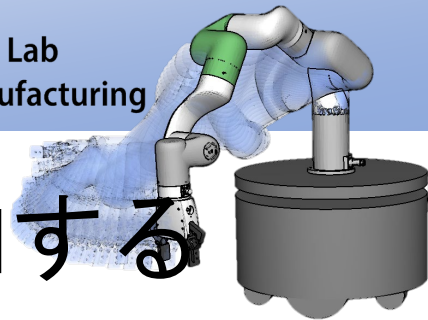


学生stの科目suの成績scを追加する

```

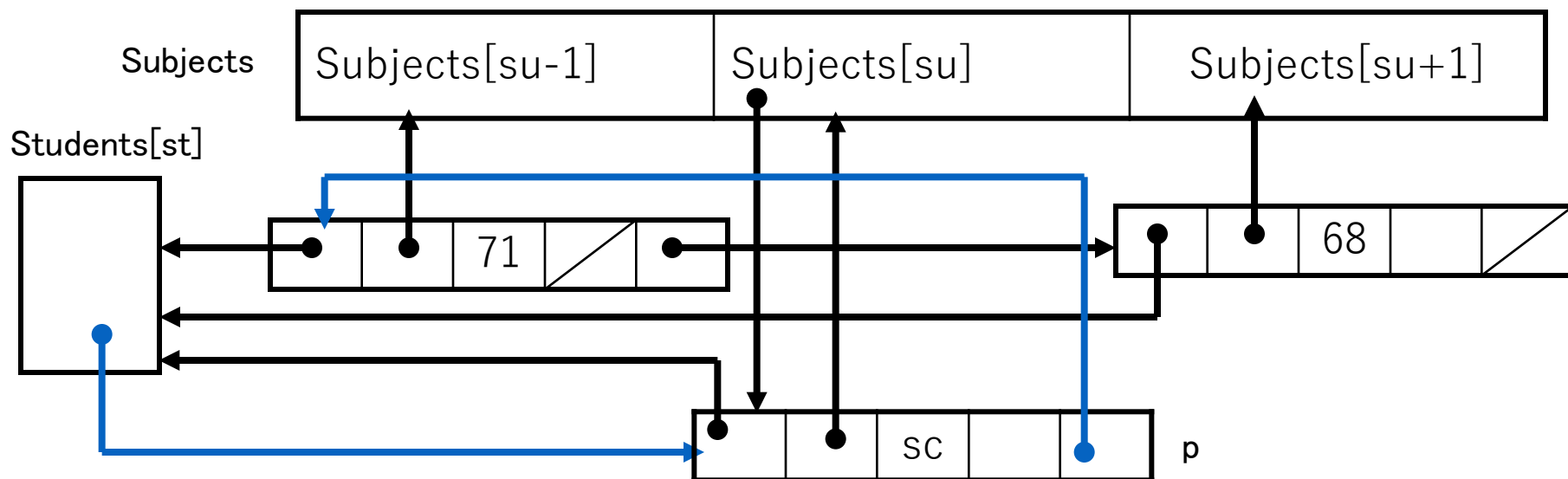
void add_credit(int st,int su,int sc) {
  struct CREDIT *p;
  if (get_score(st,su) == -1) {
    p = (struct CREDIT *)malloc(sizeof(struct CREDIT));
    p->student = &Students[st]; p->subject = &Subjects[su];
    p->score = sc;
    p->st_link = Subjects[su].st_link; Subjects[su].st_link = p;
    p->su_link = Students[st].su_link; Students[st].su_link = p;
  }
}
  
```

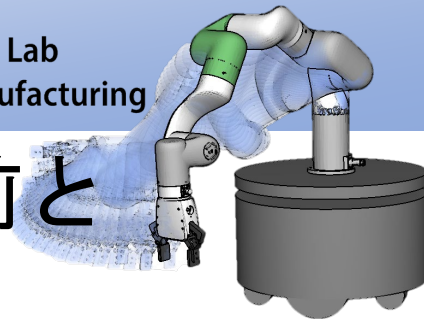




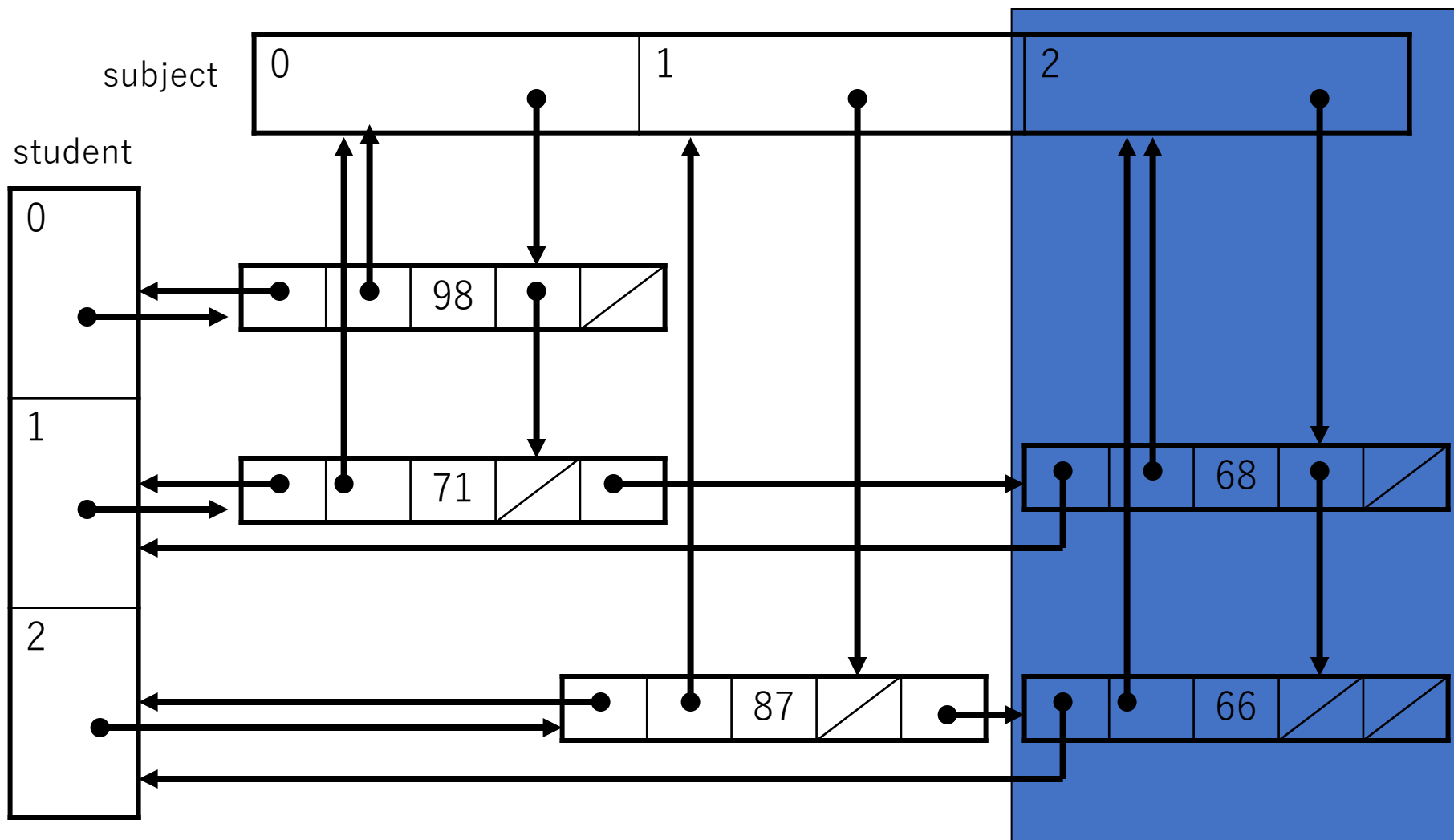
学生stの科目suの成績scを追加する

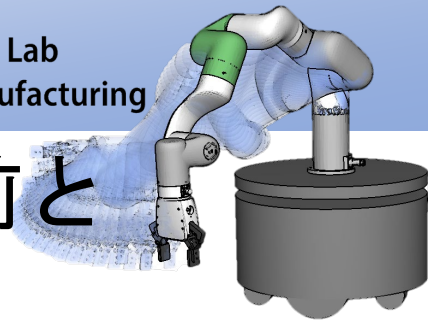
```
void add_credit(int st,int su,int sc) {
    struct CREDIT *p;
    if (get_score(st,su) == -1) {
        p = (struct CREDIT *)malloc(sizeof(struct CREDIT));
        p->student = &Students[st]; p->subject = &Subjects[su];
        p->score = sc;
        p->st_link = Subjects[su].st_link; Subjects[su].st_link = p;
        p->su_link = Students[st].su_link; Students[st].su_link = p;
    }
}
```



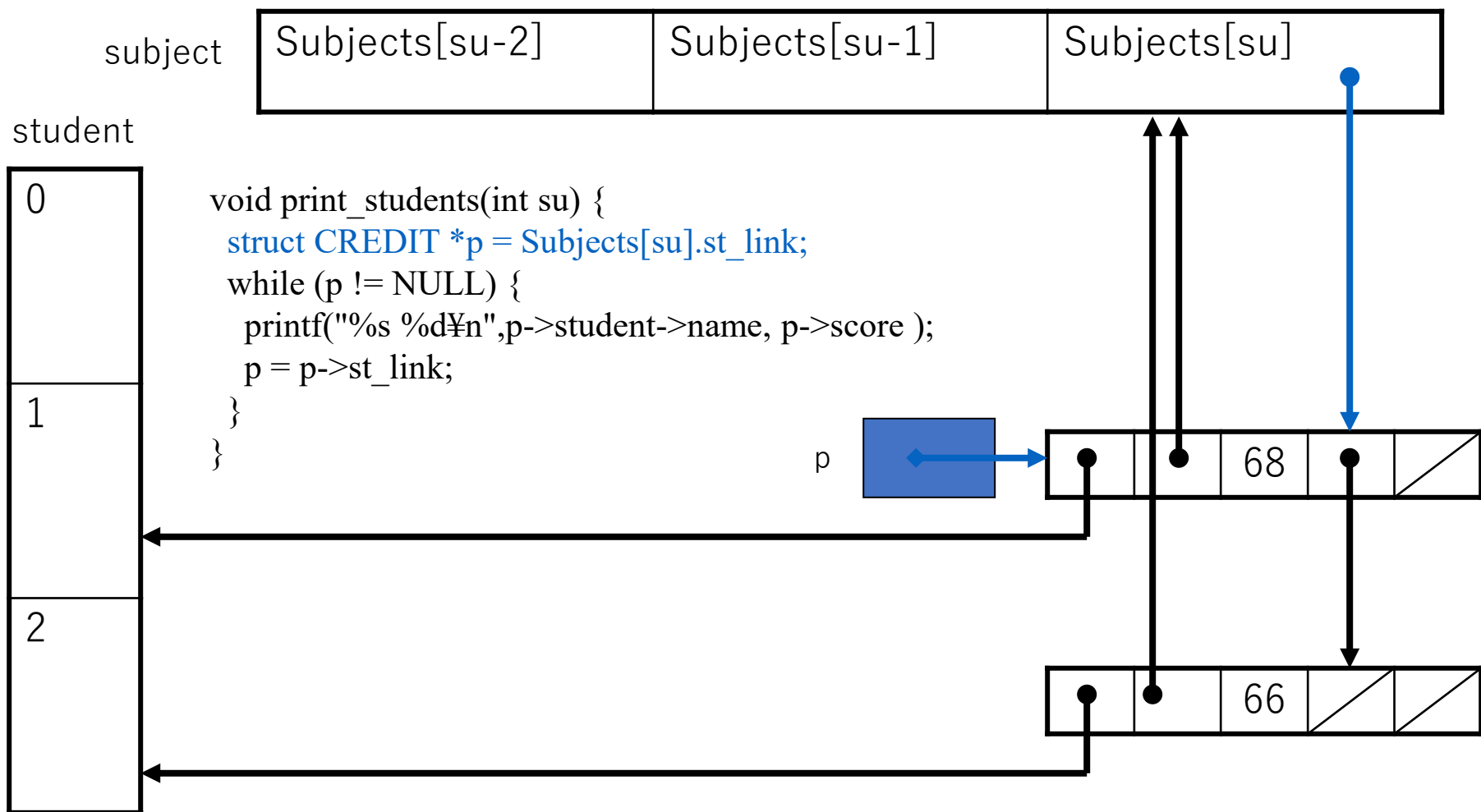


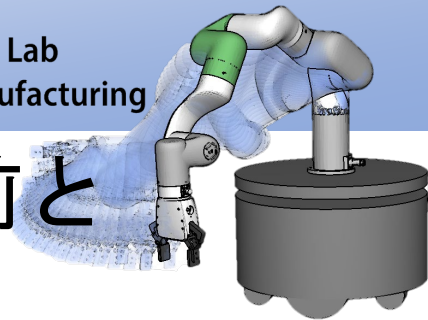
科目(su)を履修している学生の名前と 成績を表示する



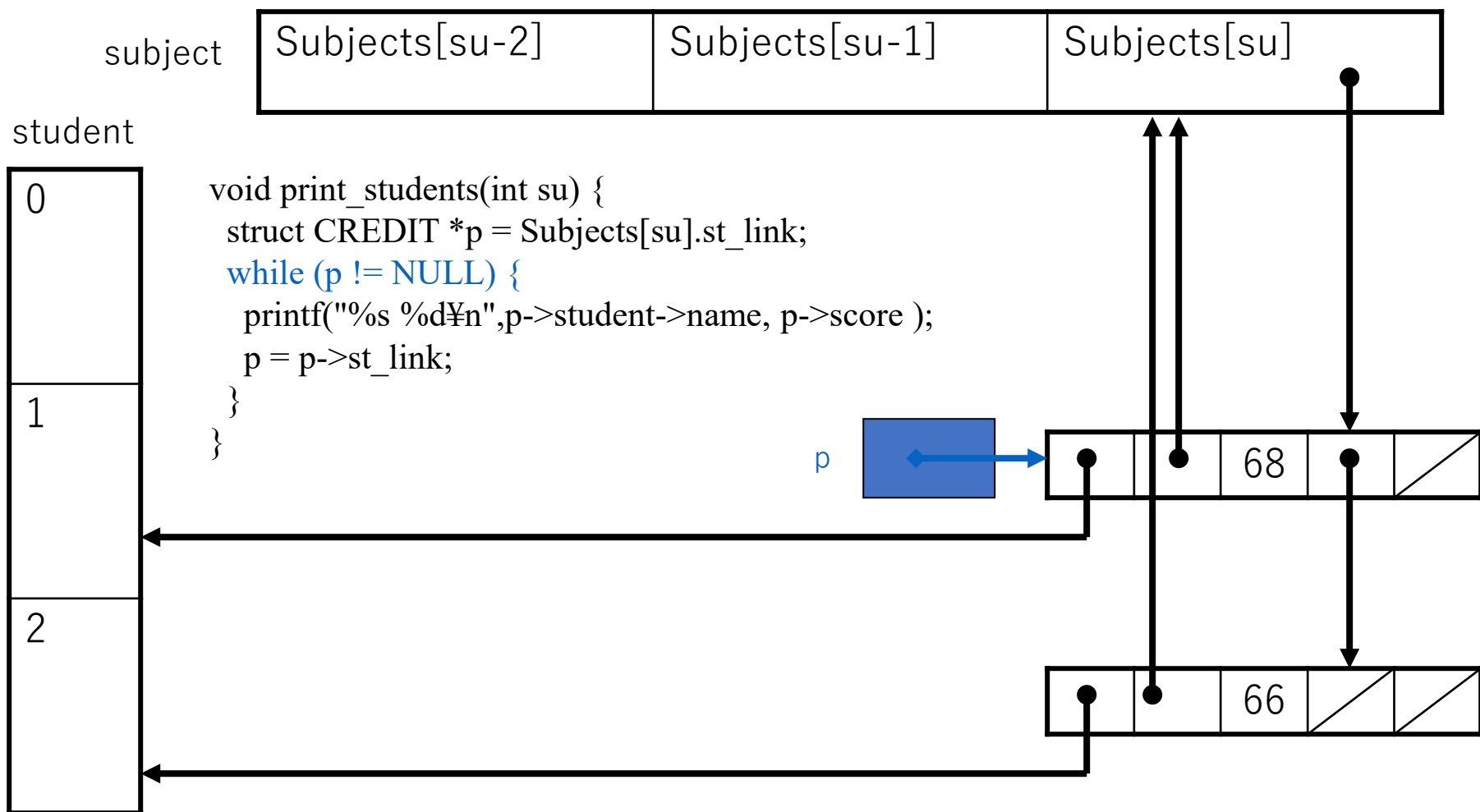


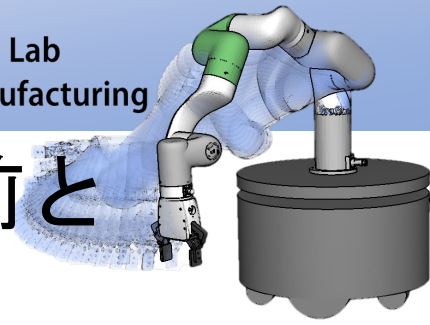
科目(su)を履修している学生の名前と成績を表示する



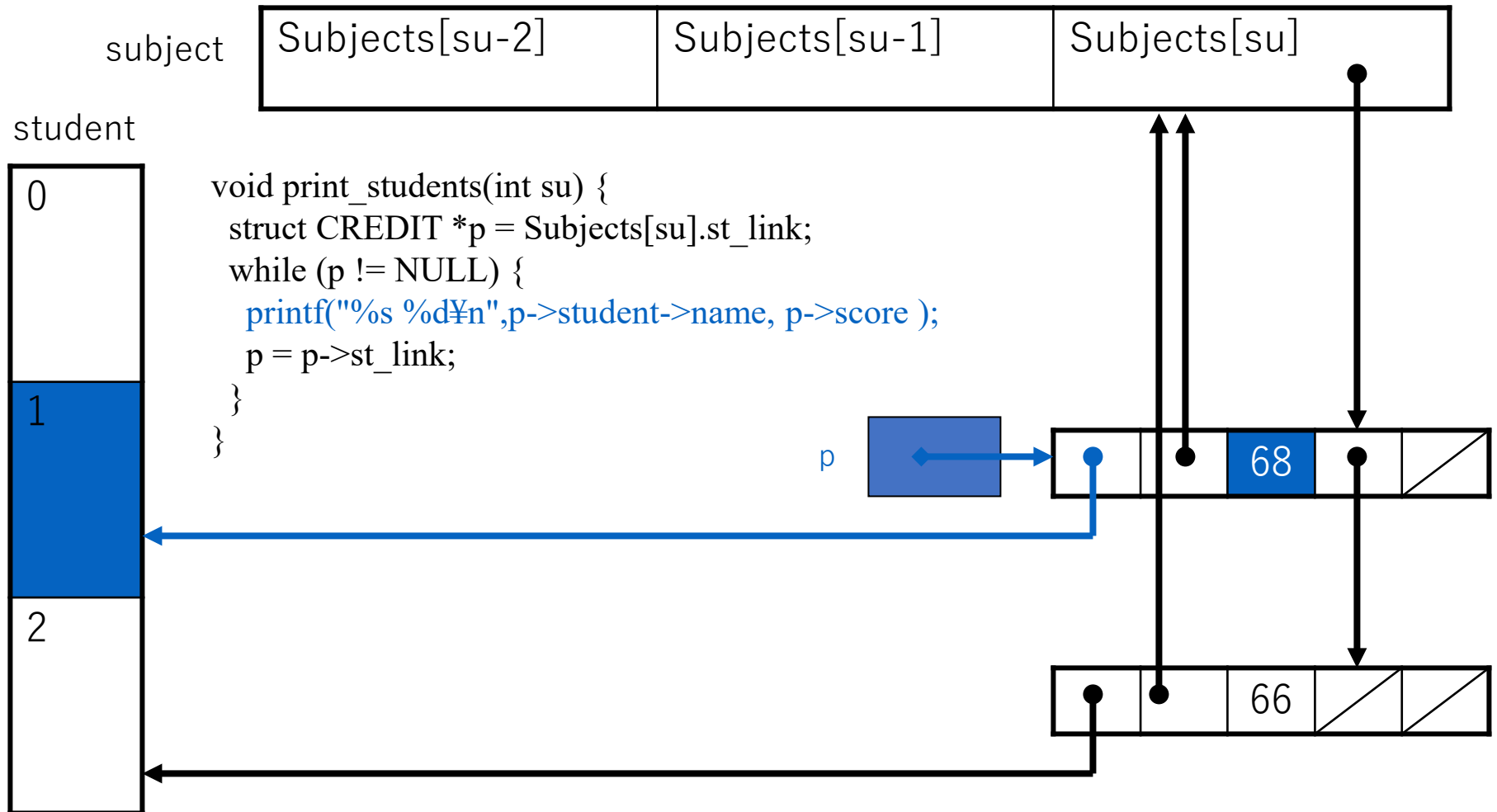


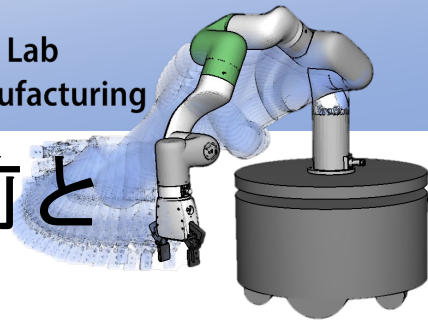
科目(su)を履修している学生の名前と成績を表示する



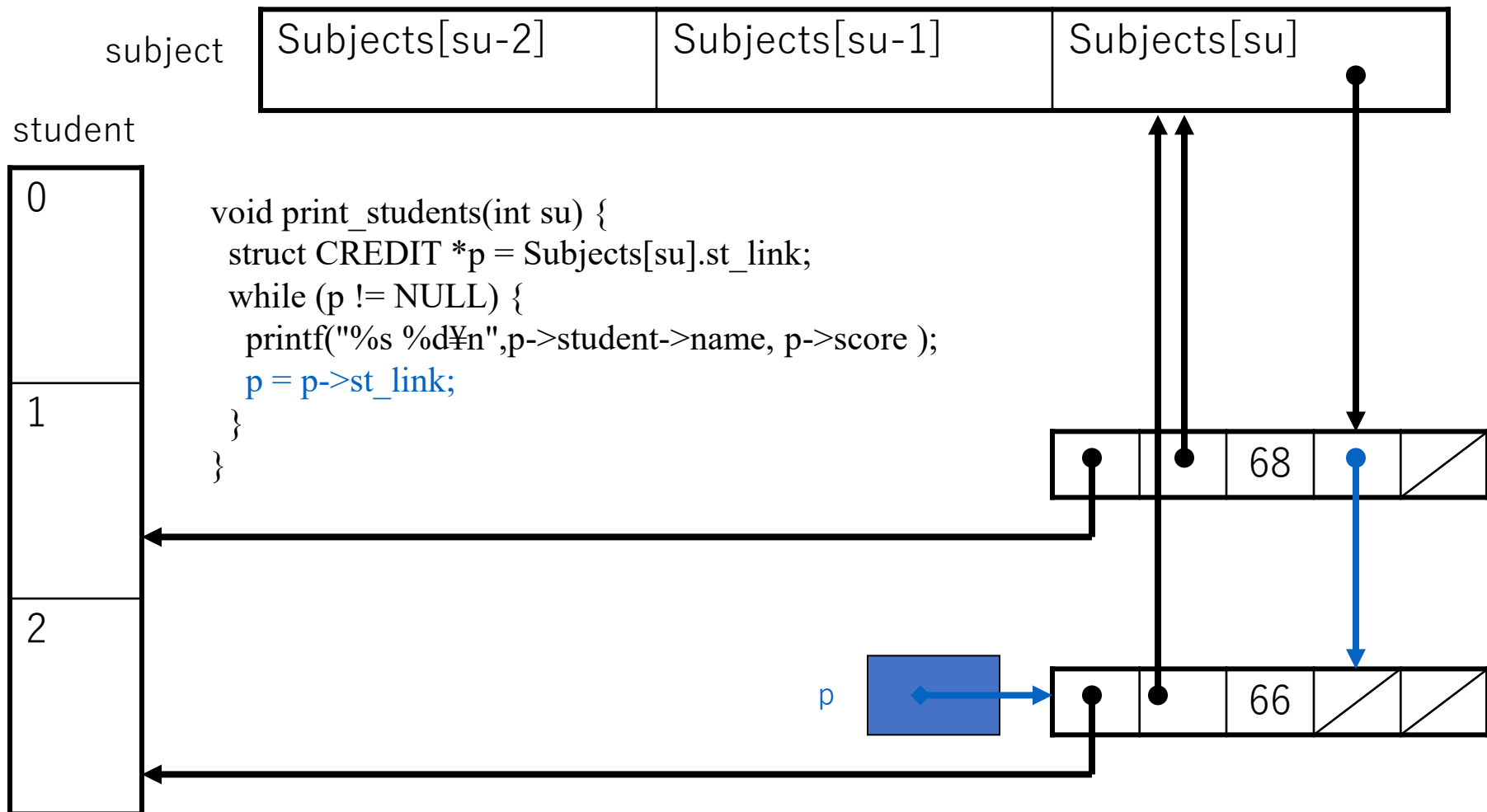


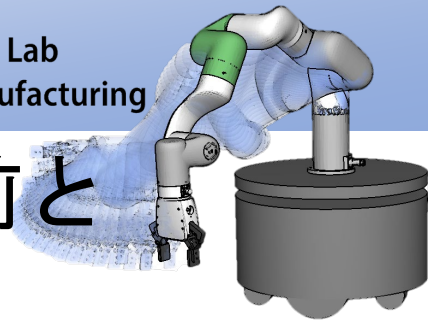
科目(su)を履修している学生の名前と成績を表示する



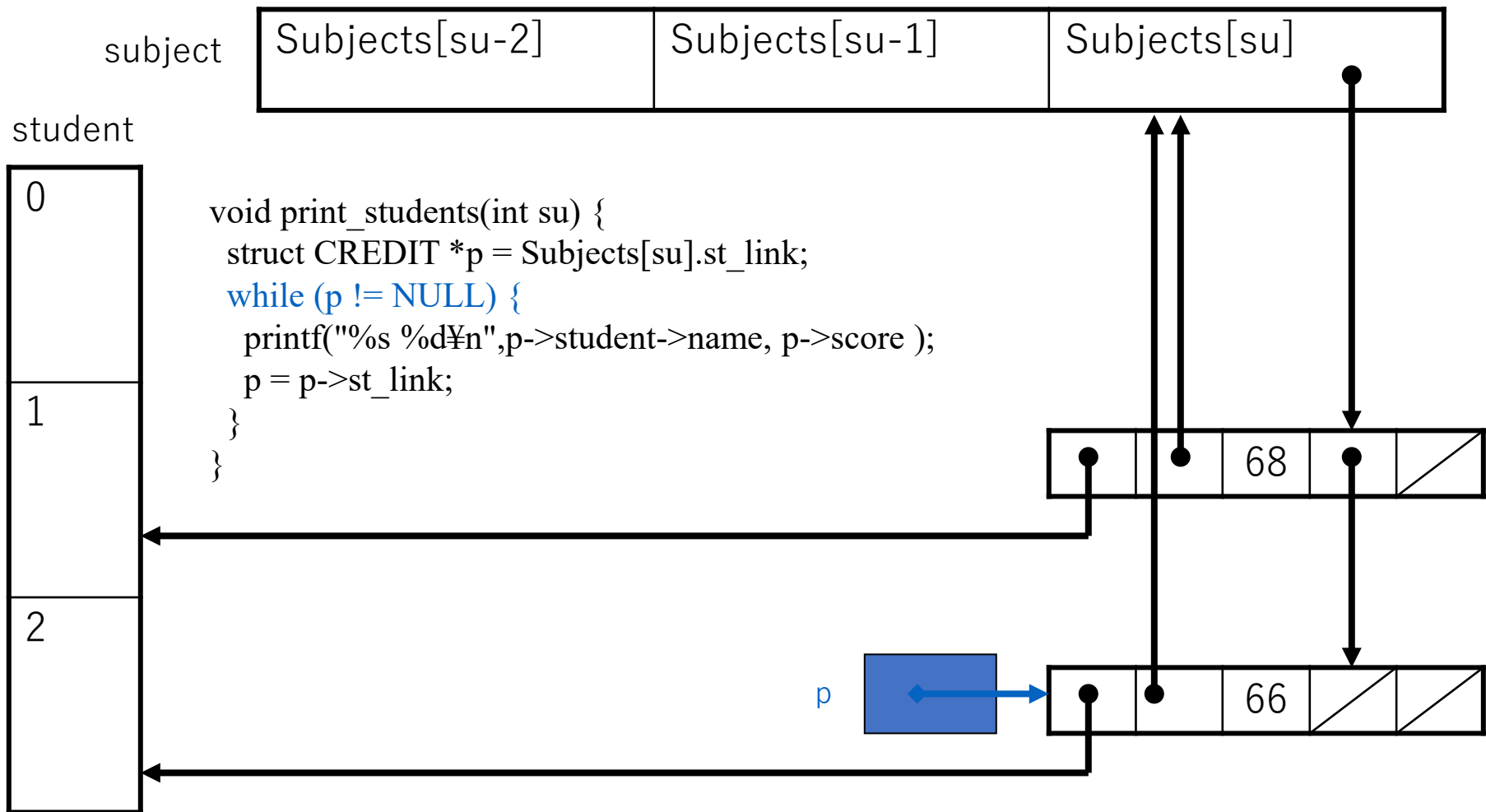


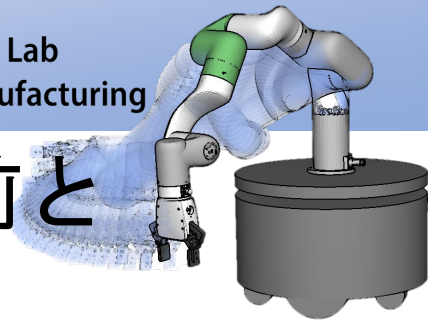
科目(su)を履修している学生の名前と 成績を表示する



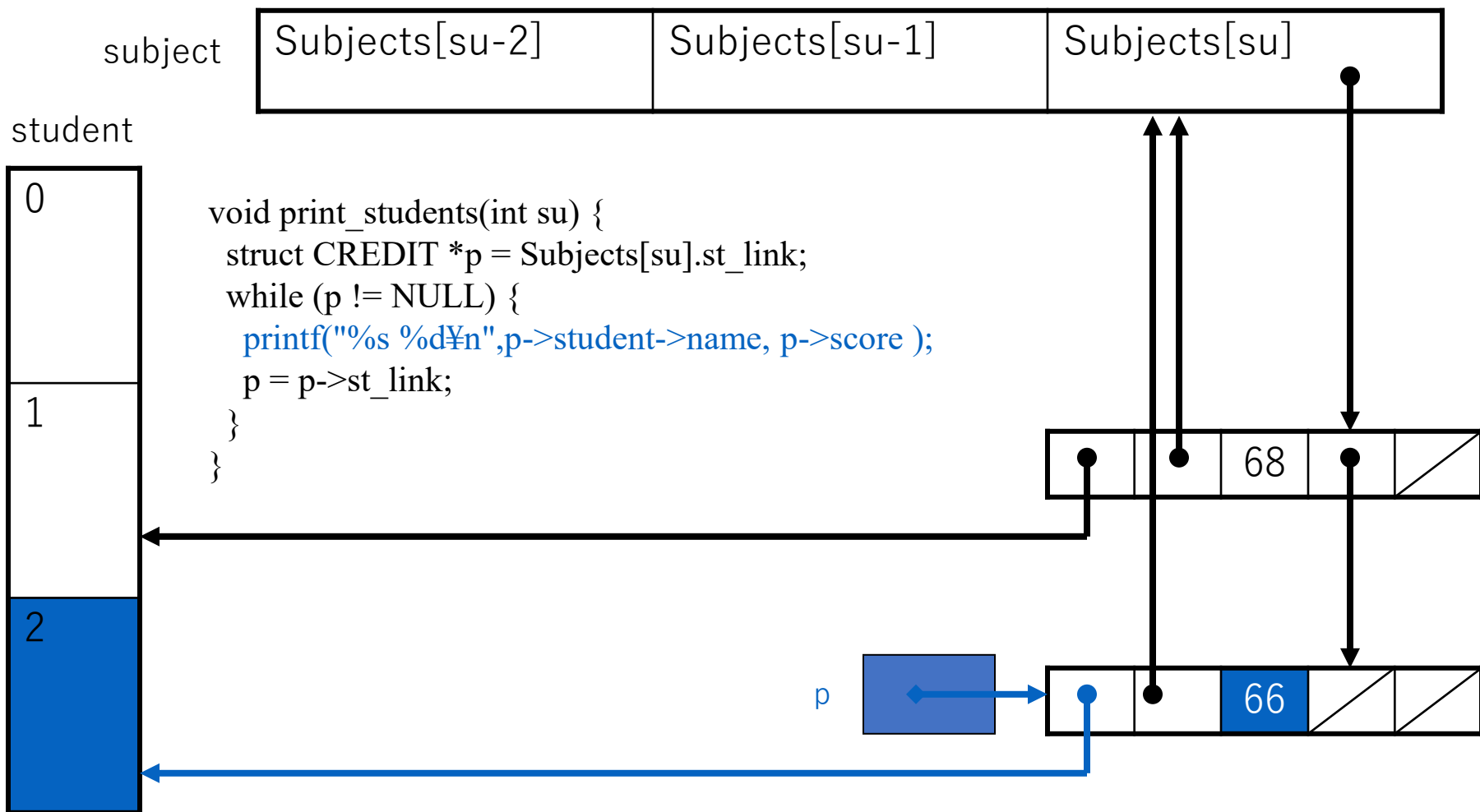


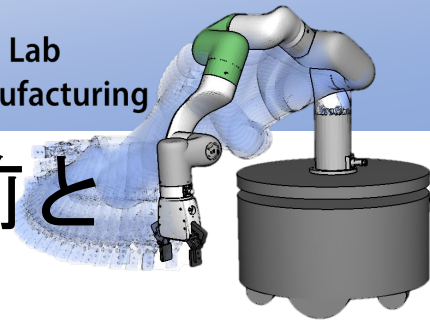
科目(su)を履修している学生の名前と 成績を表示する



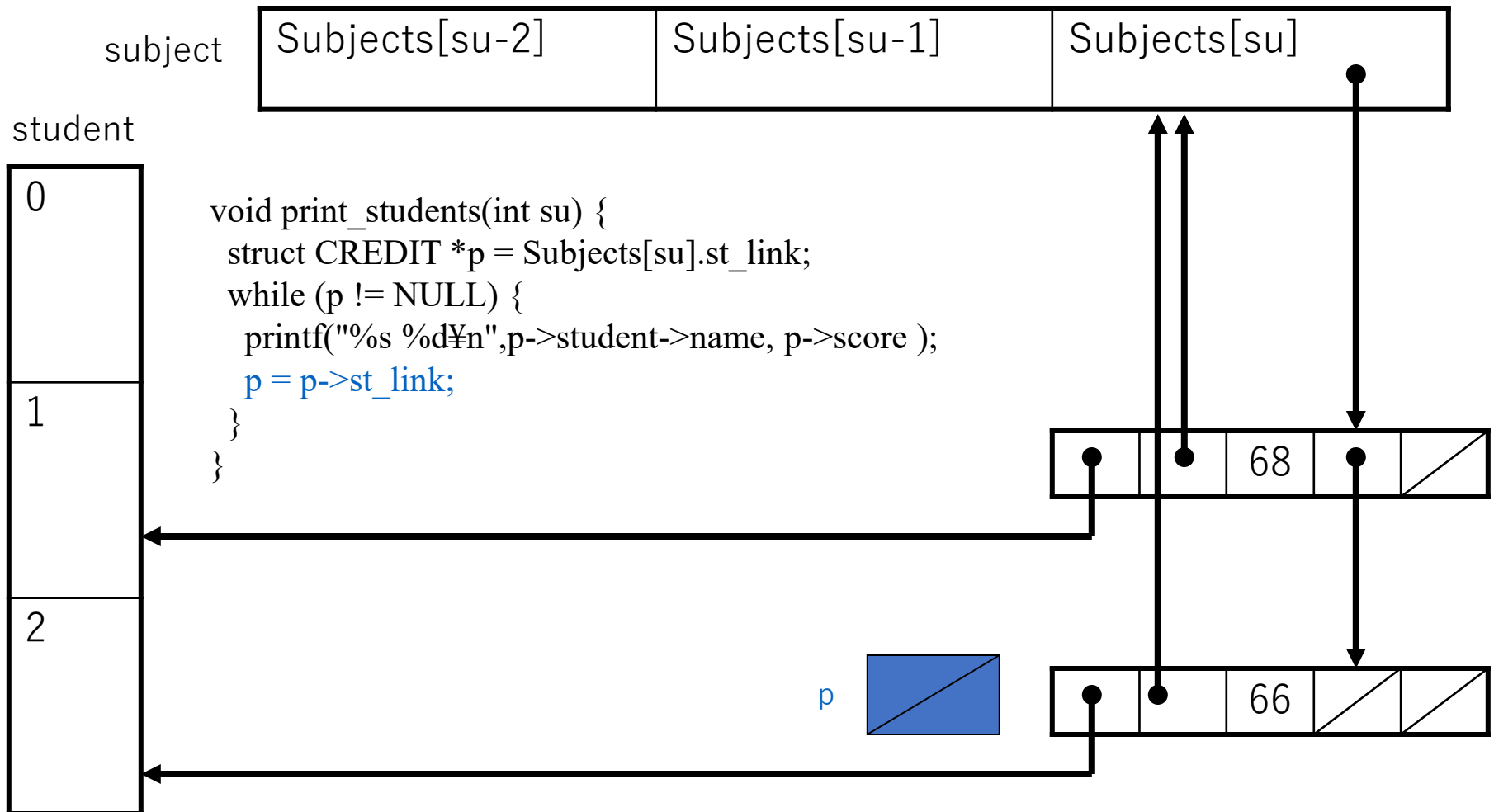


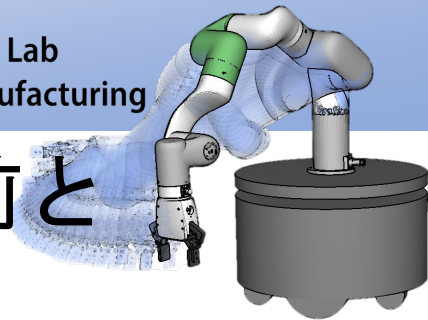
科目(su)を履修している学生の名前と 成績を表示する



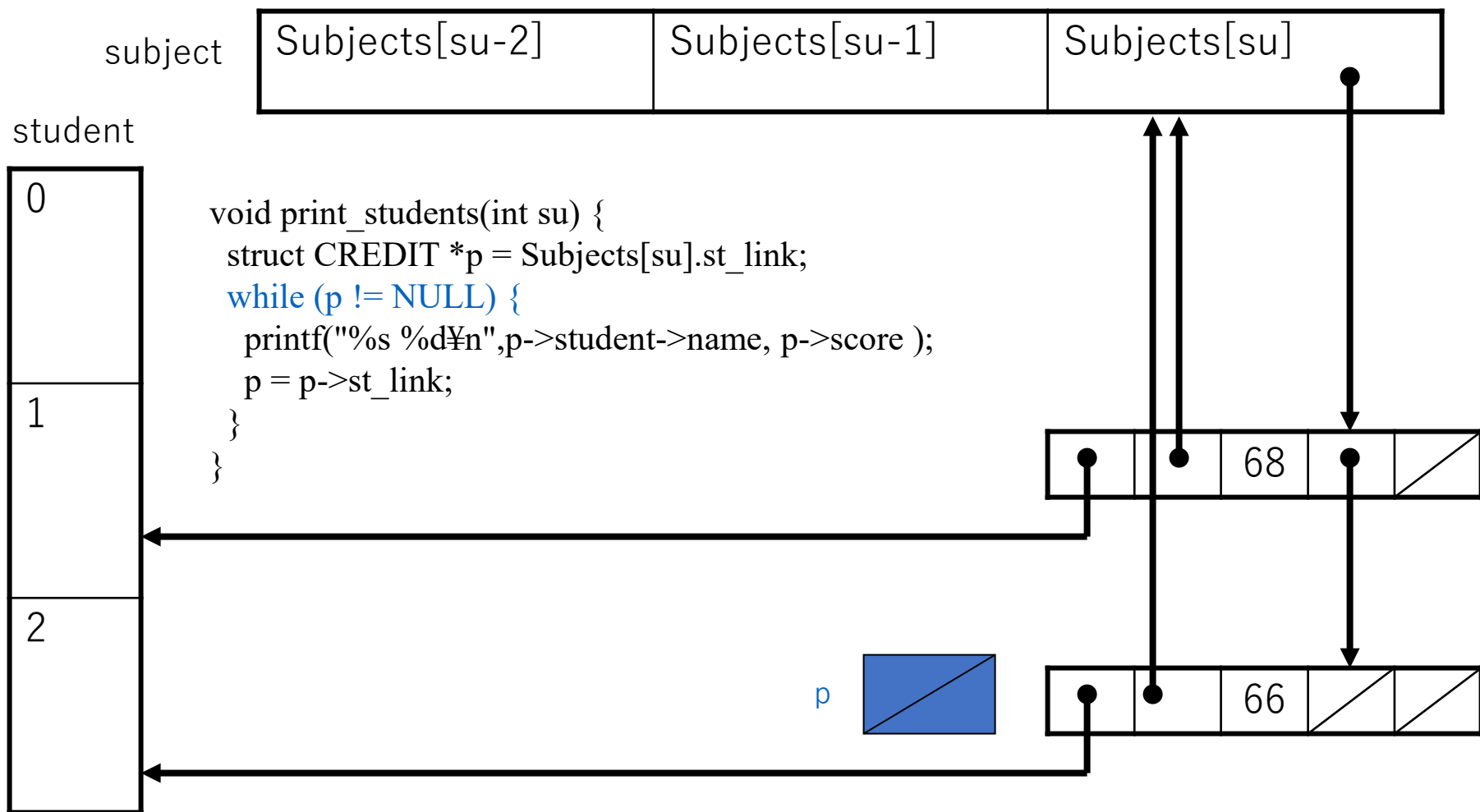


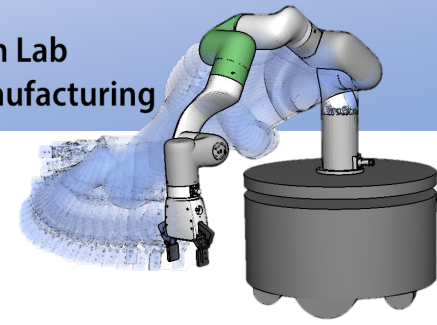
科目(su)を履修している学生の名前と 成績を表示する



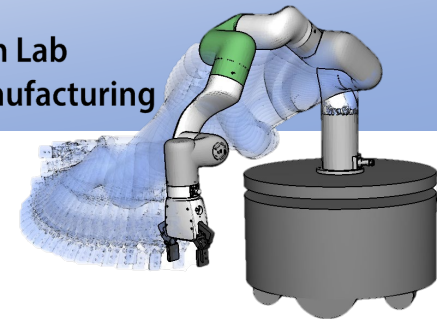


科目(su)を履修している学生の名前と 成績を表示する



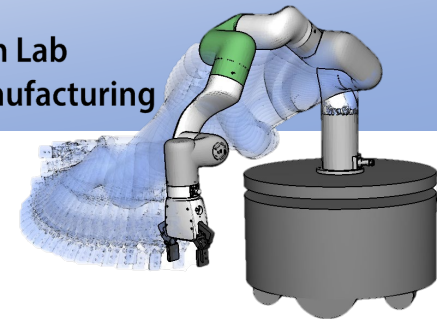


春学期の授業のまとめ



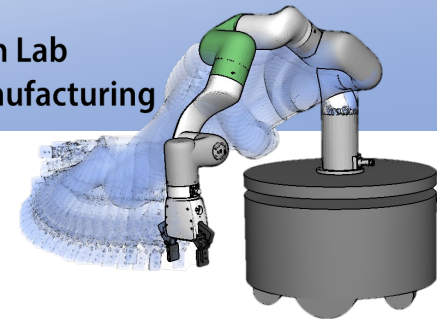
春学期の講義内容

- 1 導入：アルゴリズムとデータ構造
- 2 制御構造とプログラムの図示
- 3 配列、関数
- 4 ポインタ・文字列・構造体・ファイル
- 5 計算量・初等整列・初等探索
- 6 スタック・キュー・再帰・リスト
- 7 双方向リスト



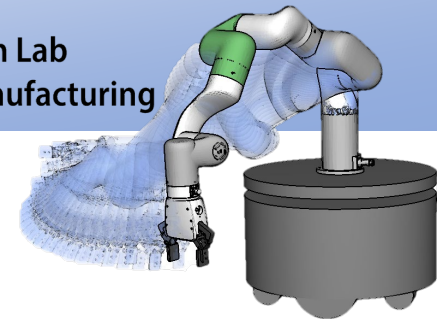
導入：アルゴリズムとデータ構造

- アルゴリズムとは
 - 問題を解くための論理または手順
- プログラミングとは
 - データ構造とアルゴリズム
 - 良いプログラム、計算方法の選択
 - 事前条件・事後条件・不変条件
 - 自然言語・疑似コーディング・Cプログラム
- データ型
 - 文字・整数・実数・配列・構造体・共用体・ポインタ
- データ構造
 - リスト、待ち行列・スタック・木・ハッシュ表・グラフ
- 演算子
 - 優先順位



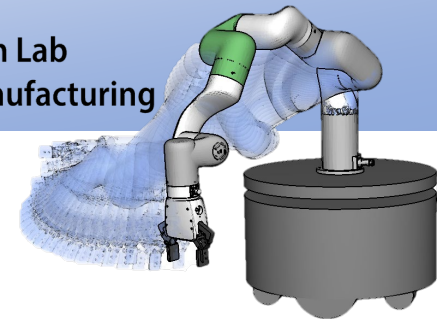
制御構造とプログラムの図示

- 基本3構造
 - 連接構造、選択構造、反復構造
 - 単一選択、分岐選択、複数選択
 - 前判定反復、後判定反復、カウンタ制御反復
 - フローチャート・NSチャートの図示
- 疑似言語
 - バッカス記法で表現すること



配列、関数

- 配列
 - 宣言、初期化（2次配列、3次配列）
 - 最大値の計算
 - 2次配列の横計、縦計、総合計の計算
- 関数
 - 宣言、仮引数、実引数、呼び出し方法、戻り値
 - 局所変数、大域変数
 - 記憶クラス：
 - static 隠す、初期化、メモリ確保
 - extern リンケージ
 - auto 大域変数に付けない
 - リンケージ：複数のファイル
 - 関数間のデータのやり取り
 - 値渡し、参照渡し、大域変数を利用、ポインタの利用



配列、関数

- extern指定子

- 分割コンパイルをする場合で、一つの大域変数を複数の翻訳単位で共通に用いる.

main.c

定義

```
#include <stdio.h>

int x=5; ← 同じメモリ領域を参照

void modify(void);

int main(void)
{
    printf("original: x=%d\n", x);
    modify();
    printf("modified: x=%d\n", x);
    return 0;
}
```

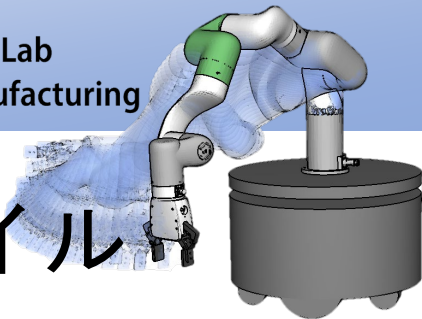
modify.c

```
extern int x;

void modify(void)
{
    x=x*2;
}
```

宣言のみ

同じ変数名の定義を2回以上行うのはおかしいため



ポインタ・文字列・構造体・ファイル

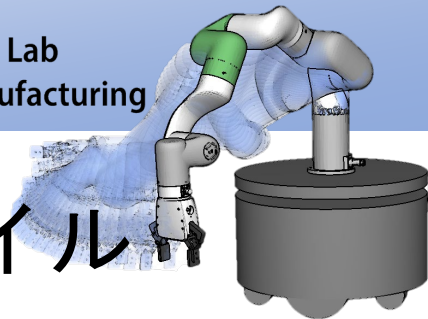
- ポインタ

- 変数の値が格納されている記憶装置の位置を指す
- 左辺値を右辺値として持つ

- ポインタの演算

- 代入
- 参照
- 前進・後進
- 等価

- ポインタは計算機アドレスと異なり，強く型に縛られている



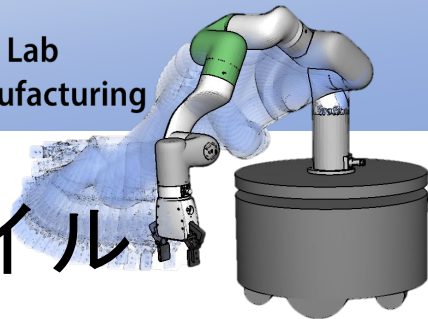
ポインタ・文字列・構造体・ファイル

- 文字列

- C言語の型ではない
 - ADTとして複数の言語に実装されている
- C言語には文字型の並びとして表現する
- 文字列の末に終端記号¥0が必要

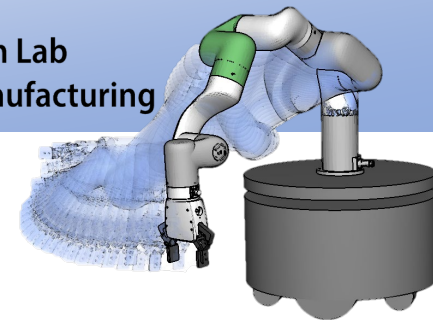
- C言語が提供している演算関数

- `char *strncpy( char *dst, char *src, size_t n );`
- `char *strncat( char *s, char *scat, size_t n );`
- `int strcmp( char *s1, char *s2);`



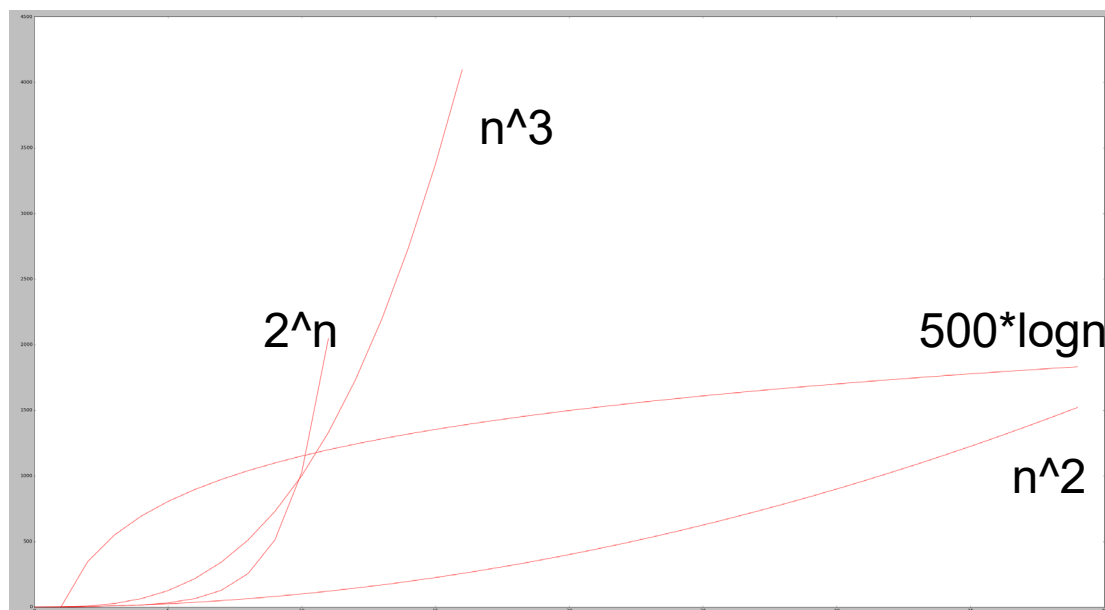
ポインタ・文字列・構造体・ファイル

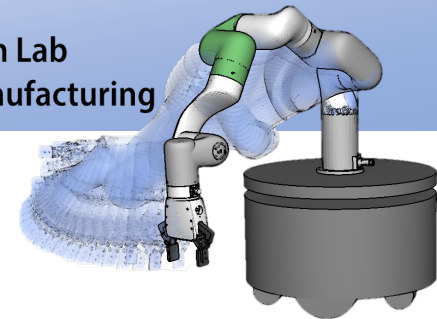
- 宣言 : struct 構造体タグ名 変数名
- ポインタ宣言 : struct 構造体タグ名 *変数名
- メンバの参照
- 構造体の配列
- 構造体ポインタのメモリ管理
 - void *malloc(size_t size)
 - void free(void *ptr)
- 構造体宣言の簡略
 - typedef struct 構造体タグ名 変数名



計算量・初等整列・初等探索

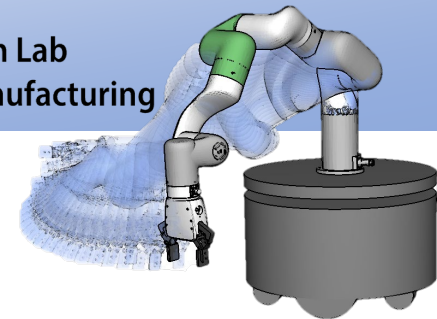
- 計算量
- 計算量の計算
 - 最大次数の項以外を除く
 - 係数を除く
- 最大次数の項
 - $O(2^n)$
 - $O(n^3)$
 - $O(n^2)$
 - $O(N \log n)$
 - $O(n)$
 - $O(\log n)$





計算量・初等整列・初等探索

- 初等的整列
- ある基準(<) でデータを並べ替えること
 - 基準(<)は各自が定義する.
- 安定な整列
- 選択ソート
 - 未整列部分から最小要素を発見し, 整列部分の後の要素とスワップ.
- バブルソート
 - 後から近接要素のみスワップ. これの2回繰り返し.
- 挿入ソート
 - 未整列部分の先頭要素を, 整列部分の適切な位置に挿入



計算量・初等整列・初等探索

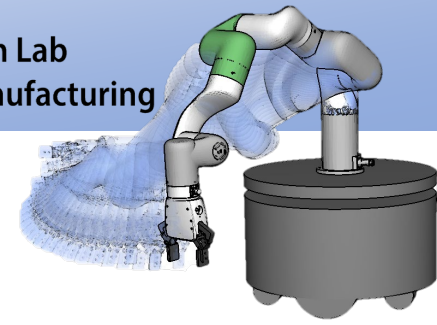
- 選択ソート

```
for (i ← 0..n-2)
  lowest ← argmin(a[j])
             i < j < n
  swap(a[i], a[lowest])
```



配列 $a[]$ の i 以降の要素
で最小値の要素番号を求める

- ループ n 回, argmin の計算量 $O(n)$
- 選択ソートの計算量は $O(n^2)$

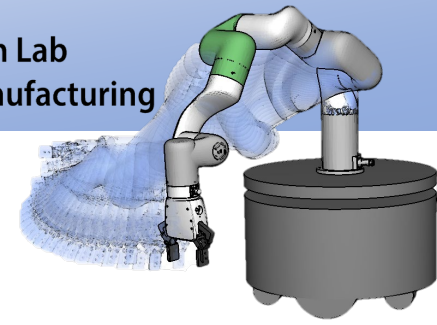


計算量・初等整列・初等探索

・バブルソート

```
for (i←0..n-1)
  for (j←n-1..i)
    if  $!(a[j-1] \leq a[j])$  swap(a[j-1],a[j])
```

右辺値の交換



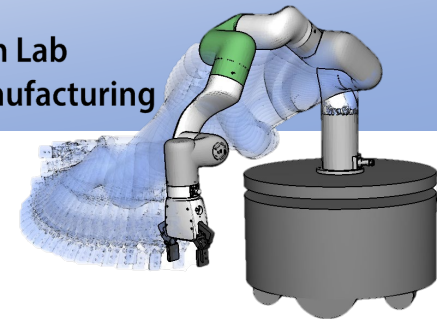
計算量・初等整列・初等探索

• 挿入ソート

```
for (i←1..n)
  j←i
  while (j>=1 && ! (a[j-1] ≤ a[j]) )
    swap( a[j-1], a[j] )
    j←j-1
  endwhile
endfor
```

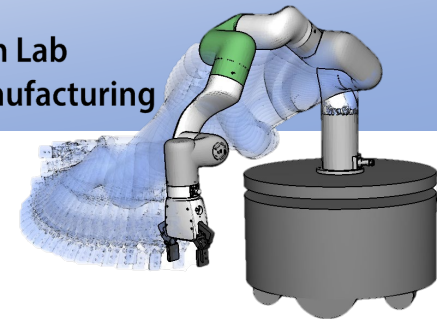
計算量：外側ループ $O(n)$
 内側ループ $O(n)$

合計： $O(n^2)$



計算量・初等整列・初等探索

- 線形探索法(linear search)
 - データを最初から順番に探索する
 - 例) {2, 4, 5, 8, 9, 11, 6, 7, 15, 20} から15を探す
最初の要素から始めて9回目→計算量 $O(n)$
- 二分探索法(binary search)
 - データをあらかじめソートしておき, 中央の要素から検索する.



前回と今回の内容

- スタック・キュー・再帰・リスト
- 双方向リスト
- 宿題→なし
- 夏学期からさらに実用的なものを勉強していく！
 - 難易度は高くなります
 - 落ち着いて読むのはおすすめです
 - AIに聞くもいい、繰り返し重複するのは身につけるコツ
 - 中村先生に交代する！万にもいつでも忌憚なく質問してください。F棟522号室/0668506381